

commitling

Una mascota pixel-art original que vive en el README de tu perfil de GitHub y crece con tus commits.

Repositorio	https://github.com/BertMarti/commitling
Web publicada	https://bertmarti.github.io/commitling/
Stack	Go 1.27 (solo biblioteca estándar) · SVG con animación CSS · GitHub Action compuesta · GitHub Actions · GitHub Pages
Fecha	1 de octubre de 2026
Autoría	Alberto Martínez, con un equipo de agentes de IA (Claude Code + OpenCode)

Contenido

1. Qué es y para quién
 2. Funcionamiento a nivel de usuario
 3. Funcionamiento a nivel técnico
 4. Flujo diario
 5. Funcionalidades
 6. Calidad, CI/CD y equipo
 7. Novedades de la noche del 30/09 al 01/10
 8. Cómo explotarlo económicamente
 9. Siguiendo pasos y acciones necesarias por ti, Alberto
- Anexo: código fuente

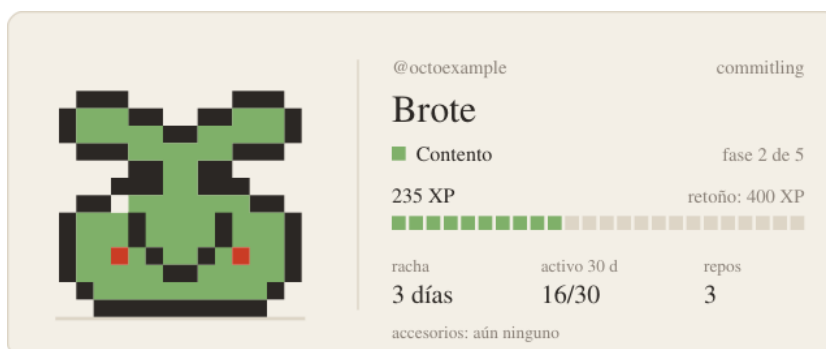
Qué es y para quién

commitling es una mascota pixel-art de diseño propio que vive en el README de tu perfil de GitHub. Una GitHub Action lee cada día tu actividad pública (commits, pull requests, issues...), calcula tu experiencia, tu racha y tus días activos, y dibuja una tarjeta SVG animada: un brote de musgo con ojos que empieza siendo una semilla y acaba convertido en un árbol ancestral.

Cambia de ánimo según lo que hayas hecho estos días y desbloquea accesorios. Convierte el perfil de GitHub en algo vivo que invita a volver y que se recuerda.

Perfil	Para qué lo usa
Desarrollador buscando empleo	Un perfil de GitHub memorable que muestra constancia de forma visual.
Estudiante	Motivación para programar un poco cada día y mantener la racha.
Creador de contenido técnico	Un elemento propio y simpático en su perfil y en sus repos.
Equipos y comunidades	Mascotas por proyecto o por equipo (idea de ampliación).

Funcionamiento a nivel de usuario



Fase Brote, contento.



Árbol ancestral radiante con los tres accesorios.

Ver cómo es

La galería <https://bertmarti.github.io/commitling/> muestra todas las fases, ánimos y accesorios en tema claro y oscuro, las reglas y la criatura en vivo de BertMarti, regenerada cada día.

Ponerla en tu perfil (5 minutos)

1. Abre o crea tu repositorio de perfil: el que se llama igual que tu usuario (tu-usuario/tu-usuario).
2. Crea el archivo `.github/workflows/commitling.yml` con el workflow que aparece en el README o en la galería (se puede copiar con un botón).
3. Añade al README la línea `![commitling](./commitling.svg)`.
4. En la pestaña **Actions**, elige «commitling» y pulsa **Run workflow**.
5. Desde entonces se actualiza sola cada día y solo hace commit cuando la criatura cambia.

Opcional: añade un segundo paso con `theme: dark` y usa un elemento `<picture>` para que GitHub muestre la versión clara u oscura según el tema de quien visita tu perfil.

Las reglas del juego

Actividad	XP
Commit (cada commit de un push)	10
Pull request abierta	25
Issue abierta	5
Cualquier otro evento público (estrellas, forks, comentarios, revisiones...)	2

Fase	Desde
Semilla	0 XP
Brote	100 XP
Retoño	400 XP
Arbusto	1.000 XP
Árbol ancestral	2.500 XP

Ánimo	Cuándo	Cómo se nota
Durmiendo	7 días o más sin actividad	Ojos cerrados, respiración lenta y «zzz»
Aburrido	De 3 a 6 días sin actividad	Párpados caídos y boca recta
Contento	Actividad en los últimos 3 días	Sonrisa, mejillas y parpadeo
Radiante	Racha de 5 días seguidos o más	Ojos felices, saltitos y destellos

Accesorio	Se desbloquea con
Gorro	Más de 30 días activos en los últimos 90
Bufanda	Racha de 10 días o más
Flor	5 repositorios distintos o más con commits, PR o issues

Funcionamiento a nivel técnico

Arquitectura

Paquete	Responsabilidad
cmd/commitling	CLI con los comandos render, gallery y version.
internal/github	Cliente de la API pública de eventos: paginación (hasta 300 eventos / 90 días), token opcional, validación de usuario y errores claros (404, límite de peticiones, 5xx).
internal/stats	Cálculo puro de XP, días activos en 30 y 90 días, racha en días UTC, días desde la última actividad y repositorios distintos. La fecha «ahora» se inyecta para poder testear.
internal/creature	Fases, ánimos y accesorios; los sprites son cuadrículas ASCII de 16×16 con una paleta de 5 colores.
internal/render	Genera la tarjeta SVG de 480×200: criatura con píxeles nítidos, ficha con fase, ánimo, XP, barra de progreso, racha y días activos; animación CSS dentro del SVG; temas claro y oscuro.
internal/gallery	Web estática con todas las variantes, las reglas y los fragmentos de instalación.
action.yml	GitHub Action compuesta: instala Go y ejecuta el CLI desde el propio repositorio de la Action.

Decisiones de diseño

- **Animación con CSS dentro del SVG:** GitHub no ejecuta JavaScript en los README, pero sí respeta las animaciones CSS de un SVG servido como imagen. Se desactivan con `prefers-reduced-motion`.
- **Determinista:** con los mismos datos sale exactamente el mismo SVG, así el repositorio del usuario solo recibe commits cuando algo cambia.
- **Solo datos públicos:** la API de eventos públicos; nada de repositorios privados.
- **Go con biblioteca estándar:** sin dependencias, compilación rápida en la Action y stack distinto a los otros proyectos.
- **Respaldo honesto:** si la API falla al desplegar la galería, se usa el fixture y la criatura aparece como «@octoexample», para no hacer pasar datos inventados por reales.
- **Diseño original:** el personaje está dibujado píxel a píxel para este proyecto; no imita criaturas de videojuegos ni marcas.

Flujo diario



Funcionalidades

Funcionalidad	Detalle
render	Genera la tarjeta de un usuario (--user) o desde un archivo de eventos (--fixture); --theme, --now y --out (también a la salida estándar).
gallery	Genera la web estática con 48 variantes, reglas, instalación y criatura en vivo.
GitHub Action	Entradas user, out, theme, token y fixture; salida path con la ruta del SVG.
5 fases	Semilla, Brote, Retoño, Arbusto y Árbol ancestral.
4 ánimos	Durmiendo, Aburrido, Contento y Radiante, cada uno con su animación.
3 accesorios	Gorro, bufanda y flor, por constancia, racha y variedad de repositorios.
Temas	Claro (papel) y oscuro (tinta), con la opción de mostrar uno u otro según el visitante.
Accesibilidad	title y desc en el SVG, sin animación para quien la desactiva en su sistema.
Web en vivo	Despliegue diario en Pages con la criatura de BertMarti actualizada.

Calidad, CI/CD y equipo

- **Tests (go test):** estadísticas (rachas con huecos, cambio de día UTC, sin eventos), umbrales exactos de fases y ánimos, accesorios, XML válido, determinismo, parser de eventos y cliente HTTP con httpptest (sin red real). Un test obliga a que el workflow del README y el de la galería sean idénticos.
- **CI** (ci.yml): gofmt, go vet, go test, render y gallery en Ubuntu y Windows.
- **Prueba de la Action** (action-test.yml): usa la propia Action con el fixture y con el usuario real.
- **CD** (deploy.yml): al actualizar main y cada día, genera la galería y la criatura en vivo y publica en Pages.
- **Git:** main protegida (solo por PR), commits convencionales y una rama por agente: agent/builder, agent/qa y agent/opcode-docs.
- **Contexto para IA:** AGENTS.md, CLAUDE.md, opcode.json y MEMORY.md, que cada agente actualiza al terminar.
- **Revisión QA** (Claude Sonnet): corrigió eventos duplicados al paginar que contaban XP dos veces, desbordes de texto con usuarios largos o XP grandes, un gris sin contraste suficiente, cabeceras y timeouts del cliente HTTP y la accesibilidad de la galería; actualizó las acciones a checkout v7 y setup-go v7.
- **Documentación** (Claude Sonnet): docs/USO.md con instalación, reglas con ejemplos, preguntas frecuentes y solución de problemas, y CONTRIBUTING.md.
- **Pull requests:** #1 builder, #2 qa (sobre builder) y #3 docs (sobre qa). Todos con el CI en verde.

Novedades de la noche del 30/09 al 01/10

- **v0.3.0:** si la API de GitHub cae, la Action conserva la última criatura, solo ante errores transitorios (5xx, 429, red); los permanentes siguen fallando a propósito.
- **v0.4.0 «Pro» (publicada, v1):** generador en vivo en <https://bertmarti.github.io/commitling/#generador>: escribes cualquier usuario y su criatura se genera en el navegador con el mismo código Go compilado a WebAssembly (4,6 MB, 1,26 MB gzip, descarga diferida), SVG idéntico byte a byte al de la CLI, descarga del SVG y workflow relleno con el usuario entre comillas.
- **Seguridad:** login validado antes de llamar a la API, criatura insertada como imagen (no como código), eventos hostiles verificados en navegador.

Cómo explotarlo económicamente

Es el proyecto con menos ingresos directos: su valor principal es la visibilidad (tu perfil y el propio repositorio pueden atraer estrellas y seguidores). Aun así hay vías:

Idea	Cómo	Esfuerzo
GitHub Marketplace	Publicar la Action en el Marketplace para que la encuentre más gente. Gratis, da visibilidad.	Bajo
GitHub Sponsors	Desbloquear especies o accesorios exclusivos para patrocinadores.	Bajo
Especies y temas premium	Nuevas criaturas originales, estaciones del año, eventos (Hacktoberfest, Navidad). Pack de pago único.	Medio
Mascotas de equipo	Una criatura por organización o repositorio que crece con la actividad del equipo; plan para empresas y comunidades.	Medio

Idea	Cómo	Esfuerzo
Merchandising	Pegatinas y camisetas con tu criatura (impresión bajo demanda).	Medio
Servicio sin Action	URL directa tipo /usuario.svg alojada, sin configurar nada; gratis con límites y de pago sin ellos.	Alto

Primer paso barato: fusionar, poner la criatura en tu propio perfil, publicar la Action en el Marketplace y compartirla en comunidades de desarrolladores con una captura animada.

Siguientes pasos y acciones necesarias por ti, Alberto

Acciones necesarias por ti

1. **Fusionar los PR en orden** (builder → qa → docs). El sistema de permisos no me deja fusionar en main. Al fusionar se publica la galería y la criatura en vivo (hasta entonces, la imagen del README sale rota).
2. Comprobar la galería: <https://bertmarti.github.io/commitling/>
3. Crear tu repositorio de perfil BertMartí/BertMartí (si no existe) y añadir el workflow y la línea del README para tener tu criatura. No lo he tocado porque es tu perfil público.
4. Crear la etiqueta v1 y una Release cuando estés conforme, y cambiar @main por @v1 en el ejemplo de instalación.
5. Opcional: publicar la Action en GitHub Marketplace (requiere aceptar sus condiciones desde tu cuenta).

Mejoras técnicas propuestas

- Más especies originales elegibles con una entrada species.
- Estadísticas de lenguajes para variar colores o accesorios.
- Modo equipo: agregar la actividad de varios usuarios u organizaciones.
- Tarjeta compacta para barras laterales y firmas.

Anexo: código fuente

Archivos principales tal y como están en main al generar este documento. Las líneas muy largas se parten y continúan en la línea siguiente.

go.mod · 4 líneas

```
module github.com/BertMarti/commitling
```

```
go 1.27.0
```

cmd/commitling/main.go · 244 líneas

```
// Command commitling draws a pixel-art creature that grows with your public
// GitHub activity.
//
// commitling render --user <login> [--theme light|dark] [--species moss|mushroom] [--keep-on-error] [--now RFC3339]
// --out commitling.svg
// commitling render --fixture testdata/events.json --out commitling.svg
// commitling gallery --out site/
// commitling og --out og.png
// commitling version
package main

import (
    "bytes"
    "context"
    "errors"
    "flag"
    "fmt"
    "io"
    "os"
    "path/filepath"
    "strings"
    "time"

    "github.com/BertMarti/commitling/internal/gallery"
    "github.com/BertMarti/commitling/internal/generate"
    "github.com/BertMarti/commitling/internal/github"
    "github.com/BertMarti/commitling/internal/og"
)

// version is the CLI version; it can be overridden with -ldflags "-X main.version=...".
var version = "0.4.0"

const usage = `commitling: una mascota pixel-art que crece con tus commits.

Uso:
    commitling render --user <usuario> [--theme light|dark] [--species moss|mushroom] [--keep-on-error] [--now RFC3339]
    --out <archivo.svg>
    commitling render --fixture <eventos.json> [--user <nombre>] [--theme light|dark] [--species moss|mushroom] [--now
    RFC3339] --out <archivo.svg>
    commitling gallery --out <directorio>
    commitling og --out <archivo.png>
    commitling version

Variables de entorno:
    GITHUB_TOKEN token opcional para la API de GitHub (más límite de peticiones)
`

// newClient builds the API client; tests replace it to point at a fake server.
var newClient = github.NewClient

func main() {
    if err := run(os.Args[1:], os.Stdout, os.Stderr); err != nil {
        fmt.Fprintln(os.Stderr, "commitling:", err)
        var ue usageError
        if errors.As(err, &ue) {
            os.Exit(2)
        }
        os.Exit(1)
    }
}

type usageError struct{ msg string }

func (e usageError) Error() string { return e.msg }

func run(args []string, stdout, stderr io.Writer) error {
    if len(args) == 0 {
        fmt.Fprint(stderr, usage)
        return usageError{"falta el subcomando"}
    }
    switch args[0] {
    case "render":
        return runRender(args[1:], stdout, stderr)
    case "gallery":
        return runGallery(args[1:], stdout, stderr)
    case "og":

```

```

        return runOG(args[1:], stdout, stderr)
    case "version", "--version", "-v":
        fmt.Fprintf(stdout, "commitling %s\n", version)
        return nil
    case "help", "--help", "-h":
        fmt.Fprint(stdout, usage)
        return nil
    }
    fmt.Fprint(stderr, usage)
    return usageError{fmt.Sprintf("subcomando desconocido: %q", args[0])}
}

func runRender(args []string, stdout, stderr io.Writer) error {
    fs := flag.NewFlagSet("render", flag.ContinueOnError)
    fs.SetOutput(stderr)
    user := fs.String("user", "", "usuario de GitHub")
    fixture := fs.String("fixture", "", "archivo JSON de eventos para trabajar sin red")
    theme := fs.String("theme", "light", "tema: light o dark")
    speciesFlag := fs.String("species", "moss", "especie: moss (brote de musgo) o mushroom (hongo)")
    nowFlag := fs.String("now", "", "fecha de referencia en RFC3339 (por defecto, ahora; con --fixture, el último evento)")
    out := fs.String("out", "commitling.svg", "archivo SVG de salida (- para la salida estándar)")
    keep := fs.Bool("keep-on-error", false, "si la API falla de forma pasajera (red, 5xx, límite de peticiones) y --out ya es un SVG, conservarlo y terminar con un aviso")
    if err := fs.Parse(args); err != nil {
        return usageError{err.Error()}
    }
    if fs.NArg() > 0 {
        return usageError{fmt.Sprintf("argumentos de más: %s", strings.Join(fs.Args(), " "))}
    }
    opts := generate.Options{User: *user, Species: *speciesFlag, Theme: *theme}
    if err := opts.Check(); err != nil {
        return usageError{err.Error()}
    }
    if *user == "" && *fixture == "" {
        return usageError{"indica --user o --fixture"}
    }

    var events []github.Event
    var err error
    if *fixture != "" {
        events, err = readFixture(*fixture)
    } else {
        ctx, cancel := context.WithTimeout(context.Background(), 3*time.Minute)
        defer cancel()
        events, err = newClient(os.Getenv("GITHUB_TOKEN")).FetchEvents(ctx, *user)
        if err != nil && *keep && github.IsTransient(err) && hasSVG(*out) {
            warnKept(stdout, stderr, *out, err)
            return nil
        }
    }
    if err != nil {
        return err
    }

    now := time.Now().UTC()
    if *fixture != "" {
        // Fixtures are frozen in time: look at them from their last event so
        // the result does not change from one day to the next.
        if latest := github.Latest(events); !latest.IsZero() {
            now = latest
        }
    }
    if *nowFlag != "" {
        now, err = time.Parse(time.RFC3339, *nowFlag)
        if err != nil {
            return usageError{fmt.Sprintf("--now no es una fecha RFC3339 válida: %v", err)}
        }
    }

    opts.Now = now
    res, err := generate.RenderEvents(events, opts)
    if err != nil {
        return err
    }
    svg, login := res.SVG, res.Login
    if *out == "-" {

```

```

        _, err = stdout.Write(svg)
        return err
    }
    if err := writeFile(*out, svg); err != nil {
        return err
    }
    fmt.Fprintf(stderr, "%s → %s (%s)\n", displayName(login), *out, res.Description)
    return nil
}

// hasSVG reports whether path already holds a finished SVG (one that ends in
// </svg>): the only thing worth keeping when the API is down.
func hasSVG(path string) bool {
    data, err := os.ReadFile(path)
    return err == nil && bytes.HasSuffix(bytes.TrimSpace(data), []byte("</svg>"))
}

// warnKept tells that the previous SVG was kept. Inside GitHub Actions it is
// also a workflow annotation (one line, on stdout, where the runner reads it).
func warnKept(stdout, stderr io.Writer, out string, cause error) {
    msg := fmt.Sprintf("la API de GitHub no responde (%v); se conserva %s sin cambios", cause, out)
    fmt.Fprintln(stderr, "commitling: aviso:", msg)
    if os.Getenv("GITHUB_ACTIONS") == "true" {
        fmt.Fprintln(stdout, "::warning title=commitling::"+strings.NewReplacer("%", "%25", "\r", "%0D", "\n", "%0A").
            Replace(msg))
    }
}

func displayName(login string) string {
    if login == "" {
        return "commitling"
    }
    return "@" + login
}

func readFixture(path string) ([]github.Event, error) {
    f, err := os.Open(path)
    if err != nil {
        return nil, err
    }
    defer f.Close()
    return github.ParseEvents(f)
}

func writeFile(path string, data []byte) error {
    if dir := filepath.Dir(path); dir != "" {
        if err := os.MkdirAll(dir, 0o755); err != nil {
            return err
        }
    }
    return os.WriteFile(path, data, 0o644)
}

func runGallery(args []string, stdout, stderr io.Writer) error {
    fs := flag.NewFlagSet("gallery", flag.ContinueOnError)
    fs.SetOutput(stderr)
    out := fs.String("out", "site", "directorío de salida")
    if err := fs.Parse(args); err != nil {
        return usageError{err.Error()}
    }
    if fs.NArg() > 0 {
        return usageError{fmt.Sprintf("argumentos de más: %s", strings.Join(fs.Args(), " "))}
    }
    n, err := gallery.Build(*out, version)
    if err != nil {
        return err
    }
    fmt.Fprintf(stderr, "galería → %s (%d archivos)\n", *out, n)
    return nil
}

func runOG(args []string, stdout, stderr io.Writer) error {
    fs := flag.NewFlagSet("og", flag.ContinueOnError)
    fs.SetOutput(stderr)
    out := fs.String("out", "og.png", "archivo PNG de salida (- para la salida estándar)")
    if err := fs.Parse(args); err != nil {
        return usageError{err.Error()}
    }

```

```

}
if fs.NArg() > 0 {
    return usageError{fmt.Sprintf("argumentos de más: %s", strings.Join(fs.Args(), " "))}
}
data, err := og.PNG(og.Hero)
if err != nil {
    return err
}
if *out == "-" {
    _, err = stdout.Write(data)
    return err
}
if err := writeFile(*out, data); err != nil {
    return err
}
fmt.Fprintf(stderr, "imagen Open Graph → %s (%dx%d)\n", *out, og.Width, og.Height)
return nil
}

```

cmd/wasm/main.go · 97 líneas

```
//go:build js && wasm

// Command wasm is the WebAssembly build of commitling: a thin layer that
// hands internal/generate to the JavaScript of the website. It has no logic of
// its own, so what the browser draws is what `commitling render` draws.
//
// GOOS=js GOARCH=wasm go build -ldflags="-s -w" -o commitling.wasm ./cmd/wasm
//
// It registers one global object, `commitling`:
//
// commitling.render(eventsJSON, user, species, theme, nowMs) -> {svg, description, login} | {error}
// commitling.workflow(user, species, theme)                    -> {workflow} | {error}
// commitling.explain(status, remaining, reset, nowMs)          -> string
// commitling.check(user, species, theme)                       -> "" when valid, else the error
//
// Empty or undefined strings mean "the default"; an empty nowMs is the clock.
package main

import (
    "syscall/js"
    "time"

    "github.com/BertMarti/commitling/internal/generate"
)

func main() {
    register()
    select {} // keep the runtime alive: JavaScript calls into it
}

func register() {
    js.Global().Set("commitling", map[string]any{
        "render": js.FuncOf(render),
        "workflow": js.FuncOf(workflow),
        "explain": js.FuncOf(explain),
        "check": js.FuncOf(check),
    })
}

// str reads an optional string argument.
func str(args []js.Value, i int) string {
    if i >= len(args) || args[i].IsUndefined() || args[i].IsNull() {
        return ""
    }
    return args[i].String()
}

// millis reads an optional time in milliseconds since the epoch (zero time
// when absent or zero).
func millis(args []js.Value, i int) time.Time {
    if i >= len(args) || args[i].Type() != js.TypeNumber || args[i].Float() == 0 {
        return time.Time{}
    }
    return time.UnixMilli(int64(args[i].Float())).UTC()
}

func options(args []js.Value, user, species, theme int) generate.Options {
    return generate.Options{User: str(args, user), Species: str(args, species), Theme: str(args, theme)}
}

func render(_ js.Value, args []js.Value) any {
    o := options(args, 1, 2, 3)
    o.Now = millis(args, 4)
    res, err := generate.Render([]byte(str(args, 0)), o)
    if err != nil {
        return map[string]any{"error": err.Error()}
    }
    return map[string]any{"svg": string(res.SVG), "description": res.Description, "login": res.Login}
}

func workflow(_ js.Value, args []js.Value) any {
    w, err := generate.Workflow(options(args, 0, 1, 2))
    if err != nil {
        return map[string]any{"error": err.Error()}
    }
    return map[string]any{"workflow": w}
}
```

```

}

func check(_ js.Value, args []js.Value) any {
    if err := options(args, 0, 1, 2).Check(); err != nil {
        return err.Error()
    }
    return ""
}

func explain(_ js.Value, args []js.Value) any {
    status := 0
    if len(args) > 0 && args[0].Type() == js.TypeNumber {
        status = args[0].Int()
    }
    now := millis(args, 3)
    if now.IsZero() {
        now = time.Now().UTC()
    }
    return generate.FetchError(status, str(args, 1), str(args, 2), now)
}

```

internal/generate/generate.go · 187 líneas

```
// Package generate is the single place where public GitHub events become a
// commitling SVG. The command line and the WebAssembly build (the live
// generator of the website) both call it, so they draw the same bytes for
// the same events and the same date.
package generate

import (
    "bytes"
    "fmt"
    "strconv"
    "strings"
    "time"

    "github.com/BertMarti/commitling/internal/creature"
    "github.com/BertMarti/commitling/internal/events"
    "github.com/BertMarti/commitling/internal/render"
    "github.com/BertMarti/commitling/internal/stats"
)

// Options say what to draw. The zero value means: the login found in the
// events, the moss sprout, the light theme and "now".
type Options struct {
    User      string // GitHub login; empty takes the most common actor of the events
    Species   string // moss or mushroom (or an alias); empty is moss
    Theme     string // light or dark; empty is light
    Now       time.Time // reference date; zero is the current time
}

// Result is what Render draws.
type Result struct {
    SVG          []byte
    Description   string // short text for alt attributes and screen readers
    Login        string // the login on the card (" " when the events have none)
}

// Check validates the options. Its messages are the ones the command line
// shows for a wrong --theme, --species or --user.
func (o Options) Check() error {
    if _, ok := render.ThemeByName(o.themeName()); !ok {
        return fmt.Errorf("tema no válido %q (usa light o dark)", o.Theme)
    }
    if _, ok := creature.SpeciesByName(o.speciesName()); !ok {
        return fmt.Errorf("especie no válida %q (usa moss o mushroom)", o.Species)
    }
    if o.User != "" && !events.ValidLogin(o.User) {
        return fmt.Errorf("nombre de usuario no válido: %q", o.User)
    }
    return nil
}

func (o Options) themeName() string {
    if o.Theme == "" {
        return "light"
    }
    return o.Theme
}

func (o Options) speciesName() string {
    if o.Species == "" {
        return "moss"
    }
    return o.Species
}

// Render draws the creature of events, a JSON array like the one returned by
// GET /users/{user}/events/public (or several pages of it joined in one array).
func Render(eventsJSON []byte, o Options) (Result, error) {
    if err := o.Check(); err != nil {
        return Result{}, err
    }
    evs, err := events.ParseEvents(bytes.NewReader(eventsJSON))
    if err != nil {
        return Result{}, err
    }
    return RenderEvents(evs, o)
}
```

```

// RenderEvents is Render for events that are already decoded.
func RenderEvents(evs []events.Event, o Options) (Result, error) {
    if err := o.Check(); err != nil {
        return Result{}, err
    }
    theme, _ := render.ThemeByName(o.themeName())
    species, _ := creature.SpeciesByName(o.speciesName())
    now := o.Now
    if now.IsZero() {
        now = time.Now().UTC()
    }
    login := o.User
    if login == "" {
        login = events.Login(evs)
    }
    card := render.NewCard(login, stats.Compute(events.Activities(evs), now), theme)
    card.Creature.Species = species
    return Result{SVG: render.SVG(card), Description: render.Description(card), Login: login}, nil
}

// BaseWorkflow is the workflow for a profile repository; the README and the
// website show this same text (a test keeps them in sync).
const BaseWorkflow = `name: commitling

on:
  schedule:
    - cron: "23 5 * * *" # cada día a las 05:23 UTC
  workflow_dispatch:

permissions:
  contents: write

jobs:
  commitling:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: BertMarti/commitling@v1
        with:
          out: commitling.svg
      - name: Guardar el SVG si ha cambiado
        run: |
          git add commitling.svg
          if git diff --cached --quiet; then
            echo "Sin cambios."
            exit 0
          fi
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git commit -m "chore: actualiza commitling"
          git push

// Workflow returns BaseWorkflow with the inputs of the Action filled in:
// user, and species and theme when they are not the defaults. Everything is
// validated first, so nothing that is not a plain login or a known value is
// ever pasted into the YAML.
func Workflow(o Options) (string, error) {
    if err := o.Check(); err != nil {
        return "", err
    }
    var inputs strings.Builder
    line := func(k, v string) { fmt.Fprintf(&inputs, "          %s: %s\n", k, v) }
    if o.User != "" {
        // Always a quoted string: YAML would read 007 as a number and null or
        // true as nothing or a boolean. A valid login only has letters, digits
        // and dashes, so Go quoting is also valid YAML quoting.
        line("user", strconv.Quote(o.User))
    }
    if sp, _ := creature.SpeciesByName(o.speciesName()); sp != creature.MossSprout {
        line("species", sp.Slug())
    }
    // The canonical name, not the raw text (spaces, capitals, line breaks).
    if theme, _ := render.ThemeByName(o.themeName()); theme.Name != render.Light.Name {
        line("theme", theme.Name)
    }
    return strings.Replace(BaseWorkflow, "          with:\n", "          with:\n"+inputs.String(), 1), nil
}
`

```

```

}

// FetchError explains in Spanish why the browser could not read the events of
// a user. status is the HTTP status (0 when there was no answer at all);
// remaining and reset are the raw X-RateLimit-Remaining and X-RateLimit-Reset
// headers ("" when absent or hidden by CORS).
func FetchError(status int, remaining, reset string, now time.Time) string {
    switch {
    case status == 0:
        return "No se pudo contactar con GitHub. Revisa tu conexión e inténtalo de nuevo."
    case status == 404:
        return "Ese usuario no existe en GitHub (404). Revisa que esté bien escrito."
    case status == 403 || status == 429:
        wait := "Vuelve a probar dentro de un rato: el contador se reinicia como mucho en una hora."
        if secs, err := strconv.ParseInt(reset, 10, 64); err == nil {
            mins := int((time.Unix(secs, 0).Sub(now) + time.Minute - 1) / time.Minute)
            if mins < 1 {
                mins = 1
            }
            unit := "minutos"
            if mins == 1 {
                unit = "minuto"
            }
            wait = fmt.Sprintf("Vuelve a probar en %d %s.", mins, unit)
        }
        return "GitHub deja hacer 60 peticiones por hora sin iniciar sesión y desde tu conexión ya se han gastado. "
    + wait +
        " La Action, con el token de tu workflow, no tiene este límite."
    case status >= 500:
        return fmt.Sprintf("GitHub no responde bien ahora mismo (%d). Inténtalo de nuevo en unos minutos.", status)
    }
    return fmt.Sprintf("GitHub respondió con el error %d. Inténtalo de nuevo más tarde.", status)
}

```

internal/events/events.go · 175 líneas

```
// Package events decodes the public events of a GitHub user and turns them
// into activities for the stats package. It does no networking (the client
// lives in internal/github), so the WebAssembly build stays small.
package events

import (
    "encoding/json"
    "fmt"
    "io"
    "sort"
    "time"

    "github.com/BertMarti/commitling/internal/stats"
)

// Event is the subset of a public GitHub event that commitling uses.
type Event struct {
    ID          string `json:"id"`
    Type        string `json:"type"`
    CreatedAt   time.Time `json:"created_at"`
    Actor       struct {
        Login string `json:"login"`
    } `json:"actor"`
    Repo struct {
        Name string `json:"name"`
    } `json:"repo"`
    Payload json.RawMessage `json:"payload"`
}

type pushPayload struct {
    Size          *int `json:"size"`
    DistinctSize *int `json:"distinct_size"`
    Commits       []json.RawMessage `json:"commits"`
}

type actionPayload struct {
    Action string `json:"action"`
}

// ParseEvents decodes a JSON array of events, like the one returned by
// GET /users/{user}/events/public or stored in testdata/events.json.
func ParseEvents(r io.Reader) ([]Event, error) {
    var events []Event
    if err := json.NewDecoder(r).Decode(&events); err != nil {
        return nil, fmt.Errorf("no se pudieron leer los eventos: %w", err)
    }
    return events, nil
}

// CommitCount returns how many commits a PushEvent carries. It prefers the
// "size" field, then the length of "commits". GitHub has been trimming push
// payloads, so a push without either still counts as one commit.
func CommitCount(e Event) int {
    var p pushPayload
    if len(e.Payload) > 0 {
        _ = json.Unmarshal(e.Payload, &p)
    }
    switch {
    case p.Size != nil && *p.Size > 0:
        return *p.Size
    case len(p.Commits) > 0:
        return len(p.Commits)
    default:
        return 1
    }
}

func action(e Event) string {
    var p actionPayload
    if len(e.Payload) > 0 {
        _ = json.Unmarshal(e.Payload, &p)
    }
    return p.Action
}

// Dedupe drops repeated events (same non-empty id), keeping the first one.
```

```

// Pages of the events API can overlap when new events arrive while paging.
func Dedupe(events []Event) []Event {
    seen := make(map[string]bool, len(events))
    out := make([]Event, 0, len(events))
    for _, e := range events {
        if e.ID != "" {
            if seen[e.ID] {
                continue
            }
            seen[e.ID] = true
        }
        out = append(out, e)
    }
    return out
}

// Activities converts events into stats activities. Repeated events (same
// id) are counted once.
//
// - PushEvent: one commit activity with Count = number of commits.
// - PullRequestEvent with action "opened" (or no action): a pull request.
// - IssuesEvent with action "opened" (or no action): an issue.
// - Anything else (stars, forks, comments, reviews, other PR/issue
//   actions...): "other" activity.
func Activities(events []Event) []stats.Activity {
    events = Dedupe(events)
    acts := make([]stats.Activity, 0, len(events))
    for _, e := range events {
        a := stats.Activity{At: e.CreatedAt, Repo: e.Repo.Name, Kind: stats.KindOther, Count: 1}
        switch e.Type {
            case "PushEvent":
                a.Kind = stats.KindCommit
                a.Count = CommitCount(e)
            case "PullRequestEvent":
                if act := action(e); act == "" || act == "opened" {
                    a.Kind = stats.KindPullRequest
                }
            case "IssuesEvent":
                if act := action(e); act == "" || act == "opened" {
                    a.Kind = stats.KindIssue
                }
        }
        acts = append(acts, a)
    }
    return acts
}

// Latest returns the time of the most recent event, or the zero time.
func Latest(events []Event) time.Time {
    var t time.Time
    for _, e := range events {
        if e.CreatedAt.After(t) {
            t = e.CreatedAt
        }
    }
    return t
}

// Login returns the most common actor login in the events, or "".
func Login(events []Event) string {
    counts := map[string]int{}
    for _, e := range events {
        if e.Actor.Login != "" {
            counts[e.Actor.Login]++
        }
    }
    logins := make([]string, 0, len(counts))
    for l := range counts {
        logins = append(logins, l)
    }
    sort.Slice(logins, func(i, j int) bool {
        if counts[logins[i]] != counts[logins[j]] {
            return counts[logins[i]] > counts[logins[j]]
        }
        return logins[i] < logins[j]
    })
    if len(logins) == 0 {

```

```

        return ""
    }
    return logins[0]
}

// ValidLogin reports whether s looks like a GitHub username: 1 to 39 ASCII
// letters, digits or dashes, not starting with a dash. It is checked by hand
// because a regexp adds half a megabyte to the WebAssembly build.
func ValidLogin(s string) bool {
    if len(s) == 0 || len(s) > 39 {
        return false
    }
    for i := 0; i < len(s); i++ {
        c := s[i]
        alnum := c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z' || c >= '0' && c <= '9'
        if !alnum && (c != '-' || i == 0) {
            return false
        }
    }
    return true
}

```

internal/github/client.go · 368 líneas

```
// Package github is the client of the public events API of GitHub: pagination,
// retries with backoff, rate-limit headers and clear errors. Decoding events
// and turning them into activities lives in internal/events (no networking, so
// the WebAssembly build does not link net/http); this package keeps aliases.
package github

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "io"
    "math"
    "net/http"
    "net/url"
    "strconv"
    "strings"
    "time"

    "github.com/BertMarti/commitling/internal/events"
)

// DefaultBaseURL is the public GitHub REST API.
const DefaultBaseURL = "https://api.github.com"

const (
    // DefaultUserAgent identifies the tool; GitHub requires a User-Agent.
    DefaultUserAgent = "commitling"
    // APIVersion is the pinned REST API version.
    APIVersion = "2022-11-28"
    // RequestTimeout bounds each HTTP request of the default client.
    RequestTimeout = 15 * time.Second
)

const (
    // MaxRetries is how many times a failed request is repeated (so a page is
    // asked for at most 1+MaxRetries times).
    MaxRetries = 3
    // DefaultBaseDelay is the first exponential-backoff wait: 1 s, 2 s, 4 s.
    DefaultBaseDelay = time.Second
    // DefaultMaxWait is the longest wait commitling accepts, whether it comes
    // from the backoff or from Retry-After / X-RateLimit-Reset. A server that
    // asks for more is not retried: better to fail with a clear message than
    // to hang a scheduled workflow.
    DefaultMaxWait = 30 * time.Second
)

const (
    perPage = 100
    // maxPages * perPage is the 300 events limit of the public events API.
    maxPages = 3
)

// ValidLogin reports whether s looks like a GitHub username.
func ValidLogin(s string) bool { return events.ValidLogin(s) }

// Client fetches public events. The zero value is not usable; use NewClient.
type Client struct {
    BaseURL    string
    Token      string
    HTTPClient *http.Client
    UserAgent  string

    // BaseDelay is the first backoff wait (default DefaultBaseDelay).
    BaseDelay time.Duration
    // MaxWait caps every wait between attempts (default DefaultMaxWait).
    MaxWait time.Duration
    // Sleep waits for d or until ctx ends; nil uses a real timer. Tests inject
    // a fake one.
    Sleep func(ctx context.Context, d time.Duration) error
    // Now gives the current time to interpret X-RateLimit-Reset and a
    // Retry-After date; nil uses time.Now.
    Now func() time.Time
}

// NewClient returns a client for the public API. token may be empty.
```

```

func NewClient(token string) *Client {
    return &Client{
        BaseURL:    DefaultBaseURL,
        Token:       token,
        HTTPClient:  &http.Client{Timeout: RequestTimeout},
        UserAgent:   DefaultUserAgent,
    }
}

// APIError is a non-successful answer from the API.
type APIError struct {
    Status int
    Message string
    // Retries is how many times the request was repeated before giving up.
    Retries int
    // RateLimited is set on a 403 that carries rate-limit headers (a plain
    // 403 is a permissions problem).
    RateLimited bool
}

// IsTransient reports whether err is a failure that may pass by itself: no
// answer at all (network), a server error (5xx) or rate limiting (429, or 403
// with rate-limit headers). A missing user (404), bad credentials (401), 422,
// a plain 403 or unparsable data will fail the same way tomorrow.
func IsTransient(err error) bool {
    var apiErr *APIError
    var netErr *net.Error
    switch {
    case errors.As(err, &apiErr):
        return apiErr.Status >= 500 || apiErr.Status == http.StatusTooManyRequests || apiErr.RateLimited
    case errors.As(err, &netErr):
        return true
    }
    return false
}

// rateLimitHeaders reports whether the headers of a 403 say it is rate limiting.
func rateLimitHeaders(hdr http.Header) bool {
    return hdr.Get("Retry-After") != "" || hdr.Get("X-RateLimit-Remaining") == "0"
}

func (e *APIError) Error() string { return e.baseError() + retriesSuffix(e.Retries) }

func (e *APIError) baseError() string {
    switch {
    case e.Status == http.StatusNotFound:
        return "usuario no encontrado en GitHub (404)"
    case e.Status == http.StatusForbidden || e.Status == http.StatusTooManyRequests:
        return fmt.Sprintf("la API de GitHub ha rechazado la petición (%d): %s; si es el límite de peticiones, define GITHUB_TOKEN", e.Status, e.Message)
    default:
        return fmt.Sprintf("la API de GitHub respondió %d: %s", e.Status, e.Message)
    }
}

// FetchEvents downloads up to 300 public events (the API maximum, about
// the last 90 days) of user, newest first.
func (c *Client) FetchEvents(ctx context.Context, user string) ([]Event, error) {
    if !ValidLogin(user) {
        return nil, fmt.Errorf("nombre de usuario no válido: %q", user)
    }
    var all []Event
    for page := 1; page <= maxPages; page++ {
        events, err := c.fetchPage(ctx, user, page)
        if err != nil {
            var apiErr *APIError
            // The API answers 422 when asking beyond its pagination limit.
            if page > 1 && errors.As(err, &apiErr) && apiErr.Status == http.StatusUnprocessableEntity {
                break
            }
            return nil, err
        }
        all = append(all, events...)
        if len(events) < perPage {
            break
        }
    }
}

```

```

    return Dedupe(all), nil
}

// netError is a failure to get any answer at all (DNS, connection refused or
// reset, timeout). It is worth repeating: the requests are all GETs.
type netError struct {
    err      error
    Retries int
}

func (e *netError) Error() string {
    return e.err.Error() + retriesSuffix(e.Retries)
}

func (e *netError) Unwrap() error { return e.err }

func retriesSuffix(n int) string {
    switch n {
    case 0:
        return ""
    case 1:
        return " (tras 1 reintento)"
    default:
        return fmt.Sprintf(" (tras %d reintentos)", n)
    }
}

// fetchPage asks for one page, repeating the request up to MaxRetries times
// on 500, 502, 503 and 504, on rate limiting (429, or 403 with rate-limit
// headers) and on network errors (unless the context has ended). Anything
// else (404, 422, other 4xx) is final.
func (c *Client) fetchPage(ctx context.Context, user string, page int) ([]Event, error) {
    for attempt := 0; ; attempt++ {
        events, hdr, err := c.doPage(ctx, user, page)
        if err == nil {
            return events, nil
        }
        var (
            apiErr *APIError
            netErr *netError
            wait   time.Duration
            ok     bool
        )
        switch {
        case errors.As(err, &apiErr):
            apiErr.Retries = attempt
            wait, ok = c.retryWait(apiErr.Status, hdr, attempt)
        case errors.As(err, &netErr) && ctx.Err() == nil:
            netErr.Retries = attempt
            wait, ok = c.backoff(attempt), true
        default:
            return nil, err
        }
        if attempt >= MaxRetries || !ok {
            return nil, err
        }
        if serr := c.sleep(ctx, wait); serr != nil {
            return nil, fmt.Errorf("%w; reintento interrumpido: %v", err, serr)
        }
    }
}

// backoff is the exponential wait before retry number attempt+1: 1x, 2x, 4x.
func (c *Client) backoff(attempt int) time.Duration {
    base := c.BaseDelay
    if base <= 0 {
        base = DefaultBaseDelay
    }
    wait := base << uint(attempt)
    if wait > c.maxWait() || wait <= 0 {
        wait = c.maxWait()
    }
    return wait
}

func (c *Client) maxWait() time.Duration {
    if c.MaxWait > 0 {

```

```

        return c.MaxWait
    }
    return DefaultMaxWait
}

// retryWait reports whether a response with this status and headers is worth
// repeating and how long to wait first.
func (c *Client) retryWait(status int, hdr http.Header, attempt int) (time.Duration, bool) {
    limited := false
    switch status {
    case http.StatusInternalServerError, http.StatusBadGateway,
        http.StatusServiceUnavailable, http.StatusGatewayTimeout:
    case http.StatusTooManyRequests:
        limited = true
    case http.StatusForbidden:
        // A plain 403 is a permissions problem; only rate limiting is retried.
        limited = rateLimitHeaders(hdr)
        if !limited {
            return 0, false
        }
    default:
        return 0, false
    }

    if hint, ok := c.serverHint(hdr, limited); ok {
        if hint > c.maxWait() {
            return 0, false // the server asks for more than we are willing to wait
        }
        return hint, true
    }
    return c.backoff(attempt), true
}

// serverHint reads how long the server asks to wait: Retry-After (seconds or
// an HTTP date) or, when rate limited, X-RateLimit-Reset (Unix seconds).
func (c *Client) serverHint(hdr http.Header, limited bool) (time.Duration, bool) {
    if v := strings.TrimSpace(hdr.Get("Retry-After")); v != "" {
        if secs, err := strconv.ParseInt(v, 10, 64); err == nil && secs >= 0 {
            return secondsToDuration(secs), true
        }
        if t, err := http.ParseTime(v); err == nil {
            return positive(t.Sub(c.now())), true
        }
    }
    if limited {
        if v := strings.TrimSpace(hdr.Get("X-RateLimit-Reset")); v != "" {
            if unix, err := strconv.ParseInt(v, 10, 64); err == nil {
                now := c.now()
                if unix > now.Unix() && unix-now.Unix() > 1<<31 {
                    return secondsToDuration(unix - now.Unix()), true // absurdly far: avoid time overflow
                }
                return positive(time.Unix(unix, 0).Sub(now)), true
            }
        }
    }
    return 0, false
}

// secondsToDuration converts without overflowing: anything beyond what a
// Duration can hold is "a very long time".
func secondsToDuration(secs int64) time.Duration {
    if secs > int64(math.MaxInt64/time.Second) {
        return time.Duration(math.MaxInt64)
    }
    return time.Duration(secs) * time.Second
}

func positive(d time.Duration) time.Duration {
    if d < 0 {
        return 0
    }
    return d
}

func (c *Client) now() time.Time {
    if c.Now != nil {
        return c.Now()
    }

```

```

    }
    return time.Now()
}

func (c *Client) sleep(ctx context.Context, d time.Duration) error {
    if c.Sleep != nil {
        return c.Sleep(ctx, d)
    }
    t := time.NewTimer(d)
    defer t.Stop()
    select {
    case <-t.C:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

func (c *Client) doPage(ctx context.Context, user string, page int) ([]Event, http.Header, error) {
    u := fmt.Sprintf("%s/users/%s/events/public?per_page=%d&page=%d",
        strings.TrimRight(c.BaseURL, "/"), url.PathEscape(user), perPage, page)
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, u, nil)
    if err != nil {
        return nil, nil, err
    }
    req.Header.Set("Accept", "application/vnd.github+json")
    req.Header.Set("X-GitHub-API-Version", APIVersion)
    ua := c.UserAgent
    if ua == "" {
        ua = DefaultUserAgent
    }
    req.Header.Set("User-Agent", ua)
    if c.Token != "" {
        req.Header.Set("Authorization", "Bearer "+c.Token)
    }

    hc := c.HTTPClient
    if hc == nil {
        hc = &http.Client{Timeout: RequestTimeout}
    }
    resp, err := hc.Do(req)
    if err != nil {
        return nil, nil, &netError{err: fmt.Errorf("no se pudo contactar con la API de GitHub: %w", err)}
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        body, _ := io.ReadAll(io.LimitReader(resp.Body, 4096))
        var msg struct {
            Message string `json:"message"`
        }
        _ = json.Unmarshal(body, &msg)
        if msg.Message == "" {
            msg.Message = http.StatusText(resp.StatusCode)
        }
        return nil, resp.Header, &APIError{Status: resp.StatusCode, Message: msg.Message,
            RateLimited: resp.StatusCode == http.StatusForbidden && rateLimitHeaders(resp.Header)}
    }
    events, err := ParseEvents(resp.Body)
    return events, resp.Header, err
}

```

internal/stats/stats.go · 161 líneas

```
// Package stats turns a list of public GitHub activities into the numbers
// that drive the creature: experience, active days, streak and so on.
//
// Everything here is pure: the reference time ("now") is always injected so
// the same input produces the same output.
package stats

import (
    "sort"
    "time"
)

// Kind classifies an activity for experience purposes.
type Kind int

const (
    // KindOther is any public event that is not a commit, PR or issue
    // (stars, forks, comments, reviews, releases...).
    KindOther Kind = iota
    // KindCommit is a single commit pushed to a public repository.
    KindCommit
    // KindPullRequest is a pull request opened.
    KindPullRequest
    // KindIssue is an issue opened.
    KindIssue
)

// Experience points per unit of each kind.
const (
    XPCommit      = 10
    XPPullRequest = 25
    XPIssue       = 5
    XPOther       = 2
)

// Window sizes in days.
const (
    // RecentWindowDays is the window used for "active days" on the card.
    RecentWindowDays = 30
    // LongWindowDays is the window the public events API covers.
    LongWindowDays = 90
)

// Activity is one public action by the user.
type Activity struct {
    At      time.Time
    Repo    string
    Kind    Kind
    Count   int // number of units (commits in a push); values < 1 count as 1
}

// Stats is the summary used by the creature and the card.
type Stats struct {
    XP          int
    Commits     int
    PullRequests int
    Issues      int
    Others      int

    // ActiveDays30 is the number of distinct UTC days with activity in the
    // last 30 days (today included).
    ActiveDays30 int
    // ActiveDays90 is the same over the last 90 days.
    ActiveDays90 int
    // Streak is the number of consecutive UTC days with activity ending
    // today, or yesterday if there is nothing yet today.
    Streak int
    // DaysSinceLast is the number of UTC days since the last activity
    // (0 = today). It is -1 when there is no activity at all.
    DaysSinceLast int
    // Repos is the number of distinct repositories with commits, pull
    // requests or issues.
    Repos int
}

// HasActivity reports whether any activity was found.
```

```

func (s Stats) HasActivity() bool { return s.DaysSinceLast >= 0 }

// Day truncates t to its UTC calendar day.
func Day(t time.Time) time.Time {
    u := t.UTC()
    return time.Date(u.Year(), u.Month(), u.Day(), 0, 0, 0, 0, time.UTC)
}

func daysBetween(from, to time.Time) int {
    return int(Day(to).Sub(Day(from)).Hours() / 24)
}

// Compute summarises the activities as seen at the instant now.
//
// Only activities inside the last 90 days (today included) and not after
// now are considered, which matches what the public events API returns.
func Compute(acts []Activity, now time.Time) Stats {
    s := Stats{DaysSinceLast: -1}
    today := Day(now)
    oldest := today.AddDate(0, 0, -(LongWindowDays - 1))

    days := map[time.Time]bool{}
    repos := map[string]bool{}

    for _, a := range acts {
        if a.At.After(now) || Day(a.At).Before(oldest) {
            continue
        }
        n := a.Count
        if n < 1 {
            n = 1
        }
        switch a.Kind {
        case KindCommit:
            s.Commits += n
            s.XP += n * XPCommit
        case KindPullRequest:
            s.PullRequests += n
            s.XP += n * XPPullRequest
        case KindIssue:
            s.Issues += n
            s.XP += n * XPIssue
        default:
            s.Others += n
            s.XP += n * XPOther
        }
        if a.Kind != KindOther && a.Repo != "" {
            repos[a.Repo] = true
        }
        days[Day(a.At)] = true
    }
    s.Repos = len(repos)
    if len(days) == 0 {
        return s
    }

    sorted := make([]time.Time, 0, len(days))
    for d := range days {
        sorted = append(sorted, d)
    }
    sort.Slice(sorted, func(i, j int) bool { return sorted[i].Before(sorted[j]) })

    last := sorted[len(sorted)-1]
    s.DaysSinceLast = daysBetween(last, today)

    for _, d := range sorted {
        ago := daysBetween(d, today)
        if ago < RecentWindowDays {
            s.ActiveDays30++
        }
        if ago < LongWindowDays {
            s.ActiveDays90++
        }
    }

    // The streak survives until the end of the day after the last activity,
    // so it does not reset in the morning before the first commit.

```

```
    if s.DaysSinceLast <= 1 {  
        for d := last; days[d]; d = d.AddDate(0, 0, -1) {  
            s.Streak++  
        }  
    }  
    return s  
}
```

internal/creature/creature.go · 261 líneas

```
// Package creature decides what the commitling looks like: its species, its
// growth stage, its mood and the accessories it has unlocked, plus the pixel
// maps used to draw it.
//
// Both species are original designs. The default one is a small moss sprout
// with eyes that grows from a honey-coloured seed into an ancient tree; the
// other one is a little mushroom that grows from a spore into a fairy ring.
package creature

import (
    "strings"

    "github.com/BertMarti/commitling/internal/stats"
)

// Species is the kind of creature. Every species has the same five stages,
// four moods and three accessories, drawn with its own pixel maps.
type Species int

// Species. MossSprout is the zero value and therefore the default.
const (
    MossSprout Species = iota
    Mushroom
)

// AllSpecies lists every species, default first.
var AllSpecies = []Species{MossSprout, Mushroom}

var speciesNames = [...]string{"Brote de musgo", "Hongo"}

var speciesSlugs = [...]string{"moss", "mushroom"}

// Names of the stages of each species, in growth order.
var speciesStageNames = [...]string{
    MossSprout: {"Semilla", "Brote", "Retoño", "Arbusto", "Árbol ancestral"},
    Mushroom:   {"Espora", "Botón", "Seta", "Seta grande", "Corro de setas"},
}

// Name is the Spanish name of the species.
func (sp Species) Name() string { return speciesNames[sp] }

// Slug is the stable ASCII identifier used in the CLI, the Action and file
// names.
func (sp Species) Slug() string { return speciesSlugs[sp] }

// StageName is the Spanish name of a stage for this species.
func (sp Species) StageName(st Stage) string { return speciesStageNames[sp][st] }

// SpeciesByName finds a species by its slug or its Spanish name, ignoring
// case and surrounding spaces. The empty string is the default species.
func SpeciesByName(name string) (Species, bool) {
    n := strings.ToLower(strings.TrimSpace(name))
    if n == "" {
        return MossSprout, true
    }
    for _, sp := range AllSpecies {
        if n == sp.Slug() || n == strings.ToLower(sp.Name()) {
            return sp, true
        }
    }
    switch n {
    case "musgo":
        return MossSprout, true
    case "seta", "setas":
        return Mushroom, true
    }
    return MossSprout, false
}

// Stage is a growth stage, unlocked by experience.
type Stage int

// Growth stages, from youngest to oldest.
const (
    Seed Stage = iota
    Sprout
)
```

```

    Sapling
    Shrub
    Ancient
)

// Experience needed to reach each stage.
const (
    SeedXP    = 0
    SproutXP   = 100
    SaplingXP  = 400
    ShrubXP    = 1000
    AncientXP  = 2500
)

// Stages lists every stage in growth order.
var Stages = []Stage{Seed, Sprout, Sapling, Shrub, Ancient}

var stageXP = [...]int{SeedXP, SproutXP, SaplingXP, ShrubXP, AncientXP}

var stageNames = [...]string{"Semilla", "Brote", "Retoño", "Arbusto", "Árbol ancestral"}

var stageSlugs = [...]string{"seed", "sprout", "sapling", "shrub", "ancient"}

// Name is the Spanish name of the stage for the default species (see
// Species.StageName for the others).
func (s Stage) Name() string { return stageNames[s] }

// Slug is a stable ASCII identifier, used for file names.
func (s Stage) Slug() string { return stageSlugs[s] }

// MinXP is the experience needed to reach the stage.
func (s Stage) MinXP() int { return stageXP[s] }

// Next returns the following stage and whether there is one.
func (s Stage) Next() (Stage, bool) {
    if s >= Ancient {
        return s, false
    }
    return s + 1, true
}

// StageFor returns the stage reached with xp experience points.
func StageFor(xp int) Stage {
    st := Seed
    for _, s := range Stages {
        if xp >= s.MinXP() {
            st = s
        }
    }
    return st
}

// Progress returns how far xp is between the current stage and the next
// one, from 0 to 1. It is 1 at the last stage.
func Progress(xp int) float64 {
    st := StageFor(xp)
    next, ok := st.Next()
    if !ok {
        return 1
    }
    span := next.MinXP() - st.MinXP()
    p := float64(xp-st.MinXP()) / float64(span)
    if p < 0 {
        return 0
    }
    return p
}

// Mood depends on recent activity.
type Mood int

// Moods, from least to most active.
const (
    Sleeping Mood = iota
    Bored
    Happy
    Radiant

```

```

)

// Mood thresholds.
const (
    // SleepingAfterDays without activity the creature falls asleep.
    SleepingAfterDays = 7
    // BoredAfterDays without activity the creature gets bored.
    BoredAfterDays = 3
    // RadiantStreak consecutive days make the creature radiant.
    RadiantStreak = 5
)

// Moods lists every mood.
var Moods = []Mood{Sleeping, Bored, Happy, Radiant}

var moodNames = [...]string{"Durmiendo", "Aburrido", "Contento", "Radiante"}

var moodSlugs = [...]string{"sleeping", "bored", "happy", "radiant"}

// Name is the Spanish name shown on the card.
func (m Mood) Name() string { return moodNames[m] }

// Slug is a stable ASCII identifier, used for file names.
func (m Mood) Slug() string { return moodSlugs[m] }

// MoodFor derives the mood from the stats:
//
// - Sleeping: no activity, or 7 or more days since the last one.
// - Bored: 3 to 6 days since the last activity.
// - Radiant: a streak of 5 or more days.
// - Happy: anything else (activity in the last 2 days).
func MoodFor(s stats.Stats) Mood {
    switch {
    case !s.HasActivity() || s.DaysSinceLast >= SleepingAfterDays:
        return Sleeping
    case s.DaysSinceLast >= BoredAfterDays:
        return Bored
    case s.Streak >= RadiantStreak:
        return Radiant
    default:
        return Happy
    }
}

// Accessories unlocked by the user.
type Accessories struct {
    Hat    bool // more than 30 active days in the last 90
    Scarf  bool // streak of 10 days or more
    Flower bool // 5 or more distinct repositories
}

// Accessory thresholds.
const (
    HatActiveDays90 = 30 // strictly more than this
    ScarfStreak     = 10
    FlowerRepos     = 5
)

// AccessoriesFor derives the unlocked accessories from the stats.
func AccessoriesFor(s stats.Stats) Accessories {
    return Accessories{
        Hat:    s.ActiveDays90 > HatActiveDays90,
        Scarf:  s.Streak >= ScarfStreak,
        Flower: s.Repos >= FlowerRepos,
    }
}

// Names returns the Spanish names of the unlocked accessories, in a fixed
// order.
func (a Accessories) Names() []string {
    var out []string
    if a.Hat {
        out = append(out, "gorro")
    }
    if a.Scarf {
        out = append(out, "bufanda")
    }
}

```

```

    if a.Flower {
        out = append(out, "flor")
    }
    return out
}

// Creature is the full state to draw.
type Creature struct {
    Species    Species
    Stage      Stage
    Mood       Mood
    Accessories Accessories
}

// StageName is the Spanish name of the current stage for its species.
func (c Creature) StageName() string { return c.Species.StageName(c.Stage) }

// FromStats derives a creature of the default species from the stats. It is
// deterministic.
func FromStats(s stats.Stats) Creature { return FromStatsAs(MossSprout, s) }

// FromStatsAs derives a creature of the given species from the stats.
func FromStatsAs(sp Species, s stats.Stats) Creature {
    return Creature{
        Species:    sp,
        Stage:      StageFor(s.XP),
        Mood:       MoodFor(s),
        Accessories: AccessoriesFor(s),
    }
}

```

internal/creature/sprites.go · 356 líneas

```
package creature

// Size is the width and height of every pixel map.
const Size = 16

// The five colours of the creature ("papel y píxel" palette).
const (
    Ink    = "#2b2724"
    Paper  = "#f3efe6"
    Moss   = "#7fb069"
    Honey  = "#e6aa68"
    Accent = "#ca3c25"
)

// Pixel codes used in the ASCII maps. '.' is transparent.
const (
    Clear byte = '.'
    Pink  byte = 'K'
    PPaper byte = 'P'
    PMoss byte = 'G'
    PHoney byte = 'H'
    PRed  byte = 'R'
)

// PaletteOrder is the drawing order of the pixel codes.
var PaletteOrder = []byte{PMoss, PHoney, PPaper, PRed, Pink}

// Color returns the hex colour of a pixel code, or "" if transparent.
func Color(p byte) string {
    switch p {
    case Pink:
        return Ink
    case PPaper:
        return Paper
    case PMoss:
        return Moss
    case PHoney:
        return Honey
    case PRed:
        return Accent
    }
    return ""
}

// Grid is a Size×Size pixel map, indexed [y][x].
type Grid [Size][Size]byte

// Point is a pixel coordinate.
type Point struct{ X, Y int }

func (g *Grid) set(x, y int, p byte) {
    if x < 0 || y < 0 || x >= Size || y >= Size || p == Clear {
        return
    }
    g[y][x] = p
}

// stamp draws an ASCII sprite with its top-left corner at (x, y).
func (g *Grid) stamp(x, y int, rows []string) {
    for dy, row := range rows {
        for dx := 0; dx < len(row); dx++ {
            g.set(x+dx, y+dy, row[dx])
        }
    }
}

func parse(rows [Size]string) Grid {
    var g Grid
    for y, row := range rows {
        for x := 0; x < Size; x++ {
            g[y][x] = row[x]
        }
    }
    return g
}

// stageArt holds the pixel map of a stage and where things go on it.
```



```

Sapling: {
  body: [Size]string{
    ".....KK.....",
    ".....KGGK.....",
    "..KK..KGGK..KK..",
    ".KGGK.KGGK.KGGK.",
    ".KGGGKKGGKKGGGK.",
    "..KKGGGGGGGGKK..",
    "....KKKGGKKK....",
    "..KKKGGGGGGKKK..",
    ".KGGGGGGGGGGGGK.",
    "KGPGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    ".KGGGGGGGGGGGGGK.",
    "..KKKKKKKKKKKK..",
    "...KHHK..KHHK...",
  },
  face:      Point{4, 8},
  hat:       Point{5, 0},
  scarfRow:  12,
  scarfTail: Point{10, 13},
  flower:    Point{12, 1},
},
Shrub: {
  body: [Size]string{
    ".....",
    "..KKK.KKKK.KKK..",
    ".KGGGKGGGKGGGK.",
    ".KGPGGGGGGGGGGK.",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGRGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    ".KGGGGGGGGGGGGGK.",
    "..KKKKKKKKKKKK..",
    "...KHK....KHK...",
    "..KHHK....KHHK..",
  },
  face:      Point{4, 6},
  hat:       Point{5, 0},
  scarfRow:  10,
  scarfTail: Point{10, 11},
  flower:    Point{11, 0},
},
Ancient: {
  body: [Size]string{
    "....KKKKKKKK....",
    "..KKGGGGGGGGGGK..",
    ".KGPGGGGGGGGGGK.",
    "KGGGGGGGGGGGGGGK",
    "KGGGRGGGGGGGGGGK",
    "KGGGGGGGGGGGGGGK",
    ".KGGGGGGGGGGGGGK.",
    "..KKHHHHHHHHHK..",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    "...KHHHHHHHHK...",
    ".KHHKHHHHHHKHHK.",
    "KKKK.KKKKK.KKKK",
  },
  face:      Point{4, 8},
  hat:       Point{5, 0},
  scarfRow:  12,
  scarfTail: Point{10, 13},
  flower:    Point{11, 1},
},
}
// Eye shapes (3x2) per mood.

```

```

var eyes = [...]string{
    Sleeping: {"...", "KKK"},
    Bored:    {"KKK", ".K."},
    Happy:    {".K.", ".K."},
    Radiant:  {".K.", "K.K"},
}

// Mouth shapes (4x2) per mood.
var mouths = [...]string{
    Sleeping: {"....", ".KK."},
    Bored:    {"...", "KKKK"},
    Happy:    {"K..K", ".KK."},
    Radiant:  {"KRRK", ".KK."},
}

var hatArt = []string{
    "..KK..",
    ".KHHK.",
    ".KRRK.",
    "KRRRRK",
    "KPPPPK",
}

var flowerArt = []string{
    ".R.",
    "RHR",
    ".R.",
}

// Sprite is a creature ready to draw, split in layers:
//
// - Outline: the silhouette of the body (always ink pixels). It is a
//   separate layer so dark themes can draw it in paper and keep the
//   creature readable, like a sticker.
// - Body: fills, mouth, blush and accessories.
// - Eyes: on their own so they can blink.
type Sprite struct {
    Outline Grid
    Body     Grid
    Eyes     Grid
}

// Blinks reports whether the eyes of this mood are open (and can blink).
func (m Mood) Blinks() bool { return m == Happy || m == Bored }

// Draw composes the sprite of a creature.
func Draw(c Creature) Sprite {
    a := artFor(c.Species, c.Stage)
    var sp Sprite
    base := parse(a.body)
    sp.Body = base
    for y := range sp.Eyes {
        for x := range sp.Eyes[y] {
            sp.Eyes[y][x] = Clear
        }
    }

    f := a.face
    sp.Eyes.stamp(f.X, f.Y, eyes[c.Mood])
    sp.Eyes.stamp(f.X+5, f.Y, eyes[c.Mood])
    sp.Body.stamp(f.X+2, f.Y+2, mouths[c.Mood])
    if c.Mood == Happy || c.Mood == Radiant {
        sp.Body.set(f.X, f.Y+2, PRed)
        sp.Body.set(f.X+7, f.Y+2, PRed)
    }

    if c.Accessories.Scarf {
        drawScarf(&sp.Body, a.scarfRow, a.scarfTail)
    }
    if c.Accessories.Flower {
        sp.Body.stamp(a.flower.X, a.flower.Y, flowerArt)
    }
    if c.Accessories.Hat {
        sp.Body.stamp(a.hat.X, a.hat.Y, hatArt)
    }

    // Split the untouched silhouette from the rest.

```

```

    for y := 0; y < Size; y++ {
        for x := 0; x < Size; x++ {
            sp.Outline[y][x] = Clear
            if base[y][x] == Pink && sp.Body[y][x] == Pink && !touched(a, c, x, y) {
                sp.Outline[y][x] = Pink
                sp.Body[y][x] = Clear
            }
        }
    }
    return sp
}

// touched reports whether an accessory covers the pixel (x, y), so an ink
// pixel there belongs to the accessory and not to the silhouette.
func touched(a stageArt, c Creature, x, y int) bool {
    in := func(p Point, rows []string) bool {
        dx, dy := x-p.X, y-p.Y
        return dy >= 0 && dy < len(rows) && dx >= 0 && dx < len(rows[dy]) && rows[dy][dx] != Clear
    }
    return (c.Accessories.Hat && in(a.hat, hatArt)) || (c.Accessories.Flower && in(a.flower, flowerArt))
}

// drawScarf wraps row y in a striped red scarf, keeping the outline at both
// ends, and lets a short tail hang from tail.
func drawScarf(g *Grid, y int, tail Point) {
    first, last := -1, -1
    for x := 0; x < Size; x++ {
        if g[y][x] != Clear {
            if first < 0 {
                first = x
            }
            last = x
        }
    }
    if first < 0 {
        return
    }
    for x := first + 1; x < last; x++ {
        if (x-first)%4 == 0 {
            g[y][x] = PPaper
        } else {
            g[y][x] = PRed
        }
    }
    g.stamp(tail.X, tail.Y, []string{"RR", "R."})
}

```

package creature

commitling · <https://bertmarti.github.io/commitling/>

```

    },
    face:      Point{4, 5},
    hat:       Point{5, 0},
    scarfRow:  12,
    scarfTail: Point{9, 13},
    flower:    Point{12, 1},
  },
  Shrub: {
    body: [Size]string{
      "...KKKKKKKK...",
      "..KKHHHHHHHHKK..",
      ".KHHPPHHHHHHK.",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHHK",
      ".KHHHHHHHHHHK.",
      "..KKKKKKKKKKK..",
      "...KPPPPPPPK...",
      "...KGGGGGK...",
      "...KGGGGGK...",
      ".KK.KGGGGGK...",
      "KHHKKGGGGGK...",
      "KKKKKKKKKKKKK.",
    },
    face:      Point{4, 5},
    hat:       Point{5, 0},
    scarfRow:  11,
    scarfTail: Point{9, 12},
    flower:    Point{11, 0},
  },
  Ancient: {
    body: [Size]string{
      "...KKKKKKKK...",
      "..KKHHHHHHHHKK..",
      ".KHHPPHHHHHHK.",
      "KHHHHHHHHHHHGGK",
      "KGGHHHHHHHHHHGK",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHHK",
      "KHHHHHHHHHHHGGK",
      ".KHHHHHHHHHHK.",
      "..KKKKKKKKKKK..",
      "...KPPPPPPPK...",
      "...KGGGGGK...",
      ".KK.KGGGGGK.KK.",
      "KHHKKGGGGGKKHHK",
      ".KGKGGGGGKKGK.",
      "KKKKKKKKKKKKKKK",
    },
    face:      Point{4, 5},
    hat:       Point{5, 0},
    scarfRow:  11,
    scarfTail: Point{9, 12},
    flower:    Point{11, 0},
  },
}

```

internal/render/render.go · 519 líneas

```
// Package render draws the commitling card as a self-contained animated SVG.
//
// The animation is plain CSS inside the SVG (GitHub does not run JavaScript
// in READMEs) and is disabled with prefers-reduced-motion. The output only
// depends on its input, so the same data always yields the same bytes.
package render

import (
    "fmt"
    "strconv"
    "strings"
    "unicode/utf8"

    "github.com/BertMarti/commitling/internal/creature"
    "github.com/BertMarti/commitling/internal/stats"
)

// Card size in pixels.
const (
    Width  = 480
    Height = 200
)

// Creature placement: a 16x16 map drawn with 10 px pixels.
const (
    pixel    = 10
    originX  = 20
    originY  = 16
    groundY  = originY + creature.Size*pixel
)

// Right-hand panel.
const (
    panelX    = 222
    panelEnd  = 460
    barCells  = 24
    barCellW  = 8
    barGap    = 2
    barHeight = 8
)

// glyphWidth is the width of one monospace character as a fraction of the
// font size. The fonts in the stack range from 0.55 (Consolas) to 0.602
// (Menlo, DejaVu Sans Mono), so 0.62 is a safe upper bound.
const glyphWidth = 0.62

// panelWidth is the horizontal room of the right-hand panel.
const panelWidth = panelEnd - panelX

// textWidth estimates the rendered width of s at the given font size.
func textWidth(s string, size float64) float64 {
    return float64(utf8.RuneCountInString(s)) * size * glyphWidth
}

// truncate cuts s so that it fits in maxW at the given size, ending in "...".
func truncate(s string, size, maxW float64) string {
    n := int(maxW / (size * glyphWidth))
    r := []rune(s)
    if len(r) <= n {
        return s
    }
    if n < 1 {
        return "..."
    }
    return string(r[:n-1]) + "..."
}

// fit picks the first candidate and size (largest first) whose estimated
// width is at most maxW. If none fits, it truncates the last candidate at
// the smallest size. It is deterministic: the same input gives the same text.
func fit(candidates []string, sizes []float64, maxW float64) (string, float64) {
    for _, c := range candidates {
        for _, sz := range sizes {
            if textWidth(c, sz) <= maxW {
                return c, sz
            }
        }
    }
}
```

```

    }
}
sz := sizes[len(sizes)-1]
return truncate(candidates[len(candidates)-1], sz, maxW), sz
}

// Theme holds the card colours. The creature keeps its own palette.
type Theme struct {
    Name string
    Bg    string
    Ink   string
    Muted string
    Line  string
    // Outline is the colour of the creature's silhouette (one of the five
    // creature colours).
    Outline string
}

// Themes.
var (
    Light = Theme{Name: "light", Bg: "#f3efe6", Ink: "#2b2724", Muted: "#6f675e", Line: "#ddd5c6", Outline: creature.
    Ink}
    Dark  = Theme{Name: "dark", Bg: "#2b2724", Ink: "#f3efe6", Muted: "#a39a8e", Line: "#4a433d", Outline: creature.
    Paper}
)

// ThemeByName returns the theme called name ("light" or "dark").
func ThemeByName(name string) (Theme, bool) {
    switch strings.ToLower(strings.TrimSpace(name)) {
    case "", "light":
        return Light, true
    case "dark":
        return Dark, true
    }
    return Theme{}, false
}

// Card is everything needed to draw.
type Card struct {
    User      string // GitHub login, shown as @user; may be empty
    Stats     stats.Stats
    Creature   creature.Creature
    Theme     Theme
}

// NewCard derives the creature from the stats.
func NewCard(user string, s stats.Stats, t Theme) Card {
    return Card{User: user, Stats: s, Creature: creature.FromStats(s), Theme: t}
}

// Escape escapes text for XML content and attribute values.
func Escape(s string) string {
    var b strings.Builder
    for _, r := range s {
        switch r {
        case '&':
            b.WriteString("&")
        case '<':
            b.WriteString("<")
        case '>':
            b.WriteString(">")
        case '"':
            b.WriteString(""")
        case '\':
            b.WriteString("'")
        default:
            // Drop characters that are not allowed in XML 1.0.
            if r == '\t' || r == '\n' || r == '\r' || (r >= 0x20 && r != 0xFFFE && r != 0xFFFF) {
                b.WriteRune(r)
            }
        }
    }
    return b.String()
}

// Thousands formats n with a dot as thousands separator (Spanish style).
func Thousands(n int) string {

```

```

    s := strconv.Itoa(n)
    neg := strings.HasPrefix(s, "-")
    if neg {
        s = s[1:]
    }
    var b strings.Builder
    for i, r := range s {
        if i > 0 && (len(s)-i)%3 == 0 {
            b.WriteByte('.')
        }
        b.WriteRune(r)
    }
    if neg {
        return "-" + b.String()
    }
    return b.String()
}

func plural(n int, one, many string) string {
    if n == 1 {
        return "1 " + one
    }
    return strconv.Itoa(n) + " " + many
}

// Description is a plain-text summary used for <desc> and alt texts.
func Description(c Card) string {
    cr := c.Creature
    s := fmt.Sprintf("%s, %s. %s XP, racha de %s, %d días activos en los últimos 30",
        cr.StageName(), strings.ToLower(cr.Mood.Name()), Thousands(c.Stats.XP),
        plural(c.Stats.Streak, "día", "días"), c.Stats.ActiveDays30)
    if acc := cr.Accessories.Names(); len(acc) > 0 {
        s += ". Accesorios: " + strings.Join(acc, ", ")
    }
    return s + "."
}

// SVG draws the card.
func SVG(c Card) []byte {
    t := c.Theme
    if t.Bg == "" {
        t = Light
    }
    cr := c.Creature
    sp := creature.Draw(cr)
    var b strings.Builder

    title := "commitling"
    if c.User != "" {
        title = "commitling de @" + c.User
    }

    fmt.Fprintf(&b, `<svg xmlns="http://www.w3.org/2000/svg" width="%d" height="%d" viewBox="0 0 %d %d" role="img"
aria-labelledby="cl-title cl-desc">`, Width, Height, Width, Height)
    b.WriteString("\n")
    fmt.Fprintf(&b, "<title id=\"cl-title\">%s</title>\n", Escape(title))
    fmt.Fprintf(&b, "<desc id=\"cl-desc\">%s</desc>\n", Escape(Description(c)))
    writeStyle(&b, c, sp)

    // Paper and frame.
    fmt.Fprintf(&b, "<rect x=\"0.5\" y=\"0.5\" width=\"%d\" height=\"%d\" rx=\"8\" fill=\"%s\" stroke=\"%s\"/>\n",
        Width-1, Height-1, t.Bg, t.Line)

    // Ground and divider.
    fmt.Fprintf(&b, "<path d=\"M28 %dh144\" stroke=\"%s\" stroke-width=\"2\" stroke-dasharray=\"8 4\" shape-
rendering=\"crispEdges\"/>\n", groundY+1, t.Line)
    fmt.Fprintf(&b, "<path d=\"M202.5 28v144\" stroke=\"%s\" stroke-dasharray=\"2 4\" shape-rendering=\"crispEdges\"/
>\n", t.Line)

    // Creature.
    b.WriteString("<g class=\"bob\" shape-rendering=\"crispEdges\">\n")
    writeGrid(&b, &sp.Outline, "", t.Outline)
    writeGrid(&b, &sp.Body, "", "")
    writeGrid(&b, &sp.Eyes, "eyes", "")
    b.WriteString("</g>\n")

    switch cr.Mood {

```

```

    case creature.Sleeping:
        writeZzz(&b, sp, t)
    case creature.Radiant:
        writeSparkles(&b, sp)
    }

    writePanel(&b, c, t)
    b.WriteString("</svg>\n")
    return []byte(b.String())
}

// SpriteSVG draws only the creature, still and cropped to its pixels. The
// silhouette is ink, or paper when the viewer prefers a dark colour scheme.
// It is used for icons and headers.
func SpriteSVG(cr creature.Creature) []byte {
    sp := creature.Draw(cr)
    x0, y0, x1, y1 := creature.Size, creature.Size, -1, -1
    for _, g := range []*creature.Grid{&sp.Outline, &sp.Body, &sp.Eyes} {
        if a, b, c, d, ok := bbox(g); ok {
            x0, y0, x1, y1 = min(x0, a), min(y0, b), max(x1, c), max(y1, d)
        }
    }
    if x1 < 0 {
        x0, y0, x1, y1 = 0, 0, creature.Size-1, creature.Size-1
    }
    w, h := (x1-x0+1)*pixel, (y1-y0+1)*pixel
    side := max(w, h)
    vx := originX + x0*pixel - (side-w)/2
    vy := originY + y0*pixel - (side-h)/2
    var b strings.Builder
    fmt.Fprintf(&b, "<svg xmlns=\"http://www.w3.org/2000/svg\" viewBox=\"%d %d %d %d\" shape-rendering=\"crispEdges\">\n", vx, vy, side, side)
    fmt.Fprintf(&b, "<style>.o path{fill:%s}@media (prefers-color-scheme:dark){.o path{fill:%s}}</style>\n", creature.
    Ink, creature.Paper)
    writeGrid(&b, &sp.Outline, "o", "")
    writeGrid(&b, &sp.Body, "", "")
    writeGrid(&b, &sp.Eyes, "", "")
    b.WriteString("</svg>\n")
    return []byte(b.String())
}

// bbox returns the bounding box (in pixels of the map) of a grid.
func bbox(g *creature.Grid) (minX, minY, maxX, maxY int, ok bool) {
    minX, minY = creature.Size, creature.Size
    maxX, maxY = -1, -1
    for y := 0; y < creature.Size; y++ {
        for x := 0; x < creature.Size; x++ {
            if g[y][x] == creature.Clear || g[y][x] == 0 {
                continue
            }
            minX, minY = min(minX, x), min(minY, y)
            maxX, maxY = max(maxX, x), max(maxY, y)
        }
    }
    return minX, minY, maxX, maxY, maxX >= 0
}

func writeStyle(b *strings.Builder, c Card, sp creature.Sprite) {
    b.WriteString("<style>\n")
    b.WriteString(`.t{font-family:ui-monospace,SFMono-Regular,Menlo,Consolas,"DejaVu Sans Mono","Liberation Mono",
monospace}` + "\n")
    b.WriteString(".b{font-weight:700}\n")

    // Whole body: breathing or hopping, depending on the mood.
    switch c.Creature.Mood {
    case creature.Radiant:
        b.WriteString(".bob{animation:hop 1.6s ease-in-out infinite}\n")
        b.WriteString("@keyframes hop{0%,55%,100%{transform:translateY(0)}25%{transform:translateY(-8px)}}\n")
    case creature.Happy:
        b.WriteString(".bob{animation:breathe 3.2s ease-in-out infinite}\n")
        b.WriteString("@keyframes breathe{0%,100%{transform:translateY(0)}50%{transform:translateY(-3px)}}\n")
    default:
        b.WriteString(".bob{animation:breathe 5.6s ease-in-out infinite}\n")
        b.WriteString("@keyframes breathe{0%,100%{transform:translateY(0)}50%{transform:translateY(-2px)}}\n")
    }

    if c.Creature.Mood.Blinks() {

```

```

        if x0, y0, x1, y1, ok := bbox(&sp.Eyes); ok {
            cx := originX + (x0+x1+1)*pixel/2
            cy := originY + (y0+y1+1)*pixel/2
            fmt.Fprintf(b, ".eyes{transform-origin:%dpx %dpx;animation:blink 4.8s infinite}\n", cx, cy)
            b.WriteString("@keyframes blink{0%,91%,97%,100%{transform:scaleY(1)}94%{transform:scaleY(.1)}}\n")
        }
    }
    switch c.Creature.Mood {
    case creature.Sleeping:
        b.WriteString(".z{animation:zz 3.6s ease-in-out infinite;opacity:.9}\n")
        b.WriteString(".z2{animation-delay:1.2s}.z3{animation-delay:2.4s}\n")
        b.WriteString("@keyframes zz{0%{opacity:0;transform:translate(0,6px)}35%{opacity:.9}100%{opacity:0;transform:translate(6px,-10px)}}\n")
    case creature.Radiant:
        b.WriteString(".sp{animation:tw 1.8s ease-in-out infinite}.sp2{animation-delay:.9s}\n")
        b.WriteString("@keyframes tw{0%,100%{opacity:.2}50%{opacity:1}}\n")
    }
    b.WriteString("@media (prefers-reduced-motion:reduce){*{animation:none!important}}\n")
    b.WriteString("</style>\n")
}

// writeGrid emits one <path> per colour, merging horizontal runs. If ink is
// not empty it replaces the colour of ink pixels.
func writeGrid(b *strings.Builder, g *creature.Grid, class, ink string) {
    var paths []string
    for _, p := range creature.PaletteOrder {
        var d strings.Builder
        for y := 0; y < creature.Size; y++ {
            for x := 0; x < creature.Size; {
                if g[y][x] != p {
                    x++
                    continue
                }
                start := x
                for x < creature.Size && g[y][x] == p {
                    x++
                }
                fmt.Fprintf(&d, "M%d %dh%dv%dh-%dz", originX+start*pixel, originY+y*pixel, (x-start)*pixel, pixel, (x-
start)*pixel)
            }
        }
        if d.Len() > 0 {
            fill := creature.Color(p)
            if p == creature.Pink && ink != "" {
                fill = ink
            }
            paths = append(paths, fmt.Sprintf("<path fill=\"%s\" d=\"%s\"/>", fill, d.String()))
        }
    }
    if len(paths) == 0 {
        return
    }
    if class != "" {
        fmt.Fprintf(b, "<g class=\"%s\">", class)
        b.WriteString(strings.Join(paths, ""))
        b.WriteString("</g>\n")
        return
    }
    b.WriteString(strings.Join(paths, "\n"))
    b.WriteString("\n")
}

// glyph draws a small pixel pattern as a single path at (x, y) with size s.
func glyph(rows []string, x, y, s int) string {
    var d strings.Builder
    for dy, row := range rows {
        for dx := 0; dx < len(row); dx++ {
            if row[dx] != '.' {
                fmt.Fprintf(&d, "M%d %dh%dv%dh-%dz", x+dx*s, y+dy*s, s, s, s)
            }
        }
    }
    return d.String()
}

var zGlyph = []string{"KKKK", "..K.", ".K..", "KKKK"}
// writeZzz floats three pixel "z" above the head of a sleeping creature.

```

```

func writeZzz(b *strings.Builder, sp creature.Sprite, t Theme) {
    _, minY, maxX, _, ok := bbox(&sp.Body)
    if !ok {
        return
    }
    x := min(originX+(maxX+1)*pixel-4, 168)
    y := max(originY+minY*pixel+6, 52)
    zs := []struct{ dx, dy, s int }{{0, 0, 2}, {12, -14, 2}, {24, -30, 3}}
    b.WriteString("<g shape-rendering=\"crispEdges\">")
    for i, z := range zs {
        fmt.Fprintf(b, "<path class=\"z z%d\" fill=\"%s\" d=\"%s\"/>", i+1, t.Ink, glyph(zGlyph, x+z.dx, y+z.dy, z.s))
    }
    b.WriteString("</g>\n")
}

var sparkle = []string{".X.", "XXX", ".X."}

// writeSparkles twinkles two little stars around a radiant creature.
func writeSparkles(b *strings.Builder, sp creature.Sprite) {
    minX, minY, maxX, _, ok := bbox(&sp.Body)
    if !ok {
        return
    }
    left := max(originX+minX*pixel-16, 6)
    right := min(originX+(maxX+1)*pixel+4, 186)
    top := max(originY+minY*pixel+4, 10)
    b.WriteString("<g shape-rendering=\"crispEdges\">")
    fmt.Fprintf(b, "<path class=\"sp\" fill=\"%s\" d=\"%s\"/>", creature.Honey, glyph(sparkle, right, top, 4))
    fmt.Fprintf(b, "<path class=\"sp sp2\" fill=\"%s\" d=\"%s\"/>", creature.Accent, glyph(sparkle, left, top+30, 4))
    b.WriteString("</g>\n")
}

func moodColor(m creature.Mood, t Theme) string {
    switch m {
    case creature.Radiant:
        return creature.Accent
    case creature.Happy:
        return creature.Moss
    case creature.Bored:
        return creature.Honey
    }
    return t.Muted
}

func text(b *strings.Builder, x, y int, size float64, fill, anchor, class, s string) {
    a := ""
    if anchor != "" {
        a = fmt.Sprintf(" text-anchor=\"%s\"", anchor)
    }
    fmt.Fprintf(b, "<text class=\"%t%s\" x=\"%d\" y=\"%d\" font-size=\"%s\" fill=\"%s\">%s</text>\n",
        class, x, y, strconv.FormatFloat(size, 'f', -1, 64), fill, a, Escape(s))
}

// fitted draws the first candidate that fits maxW, shrinking through sizes
// and finally truncating with "..." (see fit).
func fitted(b *strings.Builder, x, y int, maxW float64, sizes []float64, fill, anchor, class string, candidates ...
string) {
    s, size := fit(candidates, sizes, maxW)
    text(b, x, y, size, fill, anchor, class, s)
}

func writePanel(b *strings.Builder, c Card, t Theme) {
    cr := c.Creature
    s := c.Stats

    // Header: @user on the left and the project name on the right. A long
    // login shrinks (at most it is truncated) and then takes the whole
    // width, dropping the project name (the <title> still carries it).
    const tag = "commitling"
    if c.User == "" {
        text(b, panelX, 36, 11, t.Muted, "", "", tag)
    } else {
        user := "@" + c.User
        room := panelWidth - textWidth(tag, 11) - 12
        if textWidth(user, 11) <= room {
            text(b, panelX, 36, 11, t.Muted, "", "", user)
            text(b, panelEnd, 36, 11, t.Muted, "end", "", tag)
        }
    }
}

```

```

    } else {
        fitted(b, panelX, 36, panelWidth, []float64{11, 10, 9}, t.Muted, "", "", user)
    }
}

fitted(b, panelX, 64, panelWidth, []float64{22, 20, 18, 16}, t.Ink, "", " b", cr.StageName())

fmt.Fprintf(b, "<rect x=\"%d\" y=\"%7d\" width=\"%8\" height=\"%8\" fill=\"%s\" shape-rendering=\"crispEdges\"/>\n",
panelX, moodColor(cr.Mood, t))
text(b, panelX+14, 86, 12, t.Ink, "", "", cr.Mood.Name())
text(b, panelEnd, 86, 11, t.Muted, "end", "", fmt.Sprintf("fase %d de %d", int(cr.Stage)+1, len(creature.Stages)))

// XP row: the total goes first; the goal on the right gets what is left,
// with a shorter wording if needed.
xp, xpSize := fit([]string{Thousands(s.XP) + " XP"}, []float64{12, 11, 10, 9}, panelWidth*0.6)
text(b, panelX, 110, xpSize, t.Ink, "", "", xp)
goalRoom := panelWidth - textWidth(xp, xpSize) - 10
goalSizes := []float64{11, 10, 9}
if next, ok := cr.Stage.Next(); ok {
    fitted(b, panelEnd, 110, goalRoom, goalSizes, t.Muted, "end", "",
        fmt.Sprintf("%s: %s XP", strings.ToLower(cr.Species.StageName(next)), Thousands(next.MinXP()))),
        fmt.Sprintf("meta: %s XP", Thousands(next.MinXP())))
} else {
    fitted(b, panelEnd, 110, goalRoom, goalSizes, t.Muted, "end", "", "fase máxima", "máxima")
}

filled := int(creature.Progress(s.XP) * barCells)
if cr.Stage == creature.Ancient {
    filled = barCells
}
var on, off strings.Builder
for i := 0; i < barCells; i++ {
    x := panelX + i*(barCellW+barGap)
    seg := fmt.Sprintf("M%d 118h%dv%dh-%dz", x, barCellW, barHeight, barCellW)
    if i < filled {
        on.WriteString(seg)
    } else {
        off.WriteString(seg)
    }
}
b.WriteString("<g shape-rendering=\"crispEdges\">")
if off.Len() > 0 {
    fmt.Fprintf(b, "<path fill=\"%s\" d=\"%s\"/>", t.Line, off.String())
}
if on.Len() > 0 {
    fmt.Fprintf(b, "<path fill=\"%s\" d=\"%s\"/>", creature.Moss, on.String())
}
b.WriteString("</g>\n")

cols := []struct {
    x, w      int
    label, value string
}{
    {panelX, 86, "racha", plural(s.Streak, "día", "días")},
    {panelX + 86, 94, "activo 30 d", fmt.Sprintf("%d/30", s.ActiveDays30)},
    {panelX + 180, panelEnd - (panelX + 180), "repos", strconv.Itoa(s.Repos)},
}
for _, col := range cols {
    room := float64(col.w - 4)
    fitted(b, col.x, 150, room, []float64{10, 9}, t.Muted, "", "", col.label)
    fitted(b, col.x, 168, room, []float64{15, 13, 11, 9}, t.Ink, "", " b", col.value)
}

acc := "accesorios: aún ninguno"
if names := cr.Accessories.Names(); len(names) > 0 {
    acc = "accesorios: " + strings.Join(names, " . ")
}
fitted(b, panelX, 188, panelWidth, []float64{10, 9, 8}, t.Muted, "", "", acc)
}

```

internal/og/og.go · 187 líneas

```
// Package og draws the Open Graph preview image of the project: a 1200x630
// PNG with the creature and the name of the project.
//
// Social networks do not show SVG in link previews, so the image is drawn
// here with only the standard library (image, image/color and image/png).
// Everything is made of square pixels at an integer scale, with no
// anti-aliasing, and the result is deterministic: the same input gives the
// same bytes.
package og

import (
    "bytes"
    "image"
    "image/color"
    "image/png"

    "github.com/BertMarti/commitling/internal/creature"
)

// Image size, the recommended one for Open Graph cards.
const (
    Width  = 1200
    Height = 630
)

// Text of the card.
const (
    Name      = "commitling"
    Tagline1  = "tu mascota pixel-art"
    Tagline2  = "que crece con tus commits"
    Site      = "bertmarti.github.io/commitling"
)

// Layout, in image pixels.
const (
    heroScale = 20 // scale of the big creature
    heroX     = 96
    heroY     = 150
    textX     = 496 // left edge of the text column
    stageY    = 392 // top of the first row of small stages
    stageScale = 3
    stageGap  = 20
    stageRowH = creature.Size*stageScale + 12 // distance between rows of stages
    siteY     = 530
)

// stageRowY is the top of the row of stages of the i-th species.
func stageRowY(i int) int { return stageY + i*stageRowH }

// Palette indexes.
const (
    iPaper = iota
    iInk
    iLine
    iMuted
    iMoss
    iHoney
    iAccent
)

// palette is the "papel y píxel" one: the five colours of the creature plus
// the two greys of the card (lines and muted text, as in the SVG card).
var palette = color.Palette{
    iPaper: rgb(0xf3, 0xef, 0xe6),
    iInk:   rgb(0x2b, 0x27, 0x24),
    iLine:  rgb(0xdd, 0xd5, 0xc6),
    iMuted: rgb(0x6f, 0x67, 0x5e),
    iMoss:  rgb(0x7f, 0xb0, 0x69),
    iHoney: rgb(0xe6, 0xaa, 0x68),
    iAccent: rgb(0xca, 0x3c, 0x25),
}

func rgb(r, g, b uint8) color.Color { return color.RGBA{r, g, b, 0xff} }

// canvas is a paletted image with pixel-block drawing helpers.
type canvas struct{ img *image.Paletted }
```

```

func (c canvas) rect(x, y, w, h int, idx uint8) {
    b := c.img.Bounds()
    for py := max(y, b.Min.Y); py < min(y+h, b.Max.Y); py++ {
        for px := max(x, b.Min.X); px < min(x+w, b.Max.X); px++ {
            c.img.SetColorIndex(px, py, idx)
        }
    }
}

// text draws s with its top-left corner at (x, y); every font pixel is a
// scale*scale block.
func (c canvas) text(x, y int, s string, scale int, idx uint8) {
    for _, r := range s {
        for gy, row := range glyph(r) {
            for gx := 0; gx < glyphW; gx++ {
                if row[gx] == '#' {
                    c.rect(x+gx*scale, y+gy*scale, scale, scale, idx)
                }
            }
        }
        x += advance * scale
    }
}

func colorIndex(p byte) (uint8, bool) {
    switch p {
    case creature.PInk:
        return iInk, true
    case creature.PPaper:
        return iPaper, true
    case creature.PMoss:
        return iMoss, true
    case creature.PHoney:
        return iHoney, true
    case creature.PRed:
        return iAccent, true
    }
    return 0, false
}

// sprite draws the creature with its top-left corner at (x, y), every
// sprite pixel being a scale*scale block.
func (c canvas) sprite(x, y int, cr creature.Creature, scale int) {
    sp := creature.Draw(cr)
    for _, g := range []*creature.Grid{&sp.Outline, &sp.Body, &sp.Eyes} {
        for gy := 0; gy < creature.Size; gy++ {
            for gx := 0; gx < creature.Size; gx++ {
                if idx, ok := colorIndex(g[gy][gx]); ok {
                    c.rect(x+gx*scale, y+gy*scale, scale, scale, idx)
                }
            }
        }
    }
}

// Image draws the card. hero is the big creature on the left.
func Image(hero creature.Creature) *image.Paletted {
    img := image.NewPaletted(image.Rect(0, 0, Width, Height), palette)
    c := canvas{img}
    c.rect(0, 0, Width, Height, iPaper)

    // Frame, like the edge of a notebook page.
    const frame, inset = 6, 24
    c.rect(inset, inset, Width-2*inset, frame, iInk)
    c.rect(inset, Height-inset-frame, Width-2*inset, frame, iInk)
    c.rect(inset, inset, frame, Height-2*inset, iInk)
    c.rect(Width-inset-frame, inset, frame, Height-2*inset, iInk)

    // The creature, standing on a line.
    c.rect(heroX-20, heroY+creature.Size*heroScale, creature.Size*heroScale+40, 8, iLine)
    c.sprite(heroX, heroY, hero, heroScale)

    // Name and tagline.
    c.text(textX, 130, Name, 10, iInk)
    c.rect(textX, 234, TextWidth(Name, 10), 10, iMoss)
    c.text(textX, 282, Tagline1, 4, iInk)
    c.text(textX, 326, Tagline2, 4, iInk)
}

```

```

// The five stages of each species, one row per species, from the
// youngest to the oldest.
for i, sp := range creature.AllSpecies {
    x := textX
    for _, st := range creature.Stages {
        c.sprite(x, stageRowY(i), creature.Creature{Species: sp, Stage: st, Mood: creature.Happy}, stageScale)
        x += creature.Size*stageScale + stageGap
    }
}

c.text(textX, siteY, Site, 3, iMuted)
return img
}

// PNG returns the encoded card. The encoding is deterministic.
func PNG(hero creature.Creature) ([]byte, error) {
    var buf bytes.Buffer
    enc := png.Encoder{CompressionLevel: png.BestCompression}
    if err := enc.Encode(&buf, Image(hero)); err != nil {
        return nil, err
    }
    return buf.Bytes(), nil
}

// Hero is the creature shown on the card: a radiant sapling with a flower,
// the same as the hero of the website.
var Hero = creature.Creature{
    Stage:    creature.Sapling,
    Mood:     creature.Radiant,
    Accessories: creature.Accessories{Flower: true},
}

```

internal/gallery/gallery.go · 272 líneas

```
// Package gallery builds the static website of the project: every stage,
// mood and accessory drawn with synthetic stats, the rules, how to install
// the Action and a slot for a live creature.
package gallery

import (
    "bytes"
    _ "embed"
    "fmt"
    "html/template"
    "os"
    "path/filepath"
    "strings"

    "github.com/BertMarti/commitling/internal/creature"
    "github.com/BertMarti/commitling/internal/generate"
    "github.com/BertMarti/commitling/internal/og"
    "github.com/BertMarti/commitling/internal/render"
    "github.com/BertMarti/commitling/internal/stats"
)

// LiveUser is the user whose creature is shown live on the site.
const LiveUser = "BertMarti"

// SiteURL is where the site is published (GitHub Pages).
const SiteURL = "https://bertmarti.github.io/commitling/"

// OGImage is the preview image for social cards (Open Graph): a 1200x630 PNG,
// because social networks do not show SVG.
const OGImage = SiteURL + "og.png"

// Title and Description are used in <title>, the description meta tag and
// Open Graph.
const (
    Title      = "commitling · una mascota pixel-art para tu perfil de GitHub"
    Description = "Una mascota pixel-art original que vive en el README de tu perfil de GitHub y crece con tus"
    commits    = "commits."
)

// DemoUser is the fictitious user of the gallery cards.
const DemoUser = "octoexample"

// WorkflowSnippet is the workflow for a profile repository. The README
// shows the same text (a test keeps them in sync).
const WorkflowSnippet = generate.BaseWorkflow

// ReadmeSnippet is the line to paste in the profile README.
const ReadmeSnippet = `![commitling](./commitling.svg)`

//go:embed index.html.tmpl
var indexTmpl string

// generatorJS is the script of the live generator (see index.html.tmpl).
//
//go:embed generator.js
var generatorJS []byte

// Sample returns synthetic stats that produce exactly the given stage and
// mood with no accessories.
func Sample(st creature.Stage, m creature.Mood) stats.Stats {
    xp := 3140
    if next, ok := st.Next(); ok {
        xp = st.MinXP() + (next.MinXP() - st.MinXP()) * 45 / 100
    }
    s := stats.Stats{XP: xp, Repos: 3, ActiveDays90: 21}
    switch m {
    case creature.Sleeping:
        s.DaysSinceLast, s.Streak, s.ActiveDays30 = 9, 0, 4
    case creature.Bored:
        s.DaysSinceLast, s.Streak, s.ActiveDays30 = 4, 0, 9
    case creature.Happy:
        s.DaysSinceLast, s.Streak, s.ActiveDays30 = 0, 3, 16
    case creature.Radiant:
        s.DaysSinceLast, s.Streak, s.ActiveDays30 = 0, 7, 22
    }
    return s
}
```

```

}

// Figure is one card of the gallery.
type Figure struct {
    Light, Dark string // file paths relative to the site root
    Alt          string
    Caption      string
}

type row struct {
    Title  string
    Figures []Figure
}

// speciesSection is the part of the gallery of one species.
type speciesSection struct {
    ID      string
    Name    string
    Intro   template.HTML
    Rows    []row
    Extras  []Figure
}

type rule struct{ Name, When string }

// choice is one option of a radio group of the generator.
type choice struct{ Value, Label string }

type page struct {
    Version    string
    Title      string
    Description string
    SiteURL    string
    OGImage    string
    LiveUser   string
    Favicon    string
    Hero       string
    Species    []speciesSection
    GenSpecies []choice
    XP         []rule
    Stages     []rule
    Moods      []rule
    Acc        []rule
    Workflow   string
    Readme     string
    ReadmeDrk  string
}

// Build writes the site into dir and returns how many files it wrote.
func Build(dir, version string) (int, error) {
    n := 0
    write := func(rel string, data []byte) error {
        path := filepath.Join(dir, filepath.FromSlash(rel))
        if err := os.MkdirAll(filepath.Dir(path), 0o755); err != nil {
            return err
        }
        n++
        return os.WriteFile(path, data, 0o644)
    }
    card := func(name string, sp creature.Species, s stats.Stats) (Figure, error) {
        f := Figure{Light: "svg/" + name + ".svg", Dark: "svg/" + name + "-dark.svg"}
        lc := render.NewCard(DemoUser, s, render.Light)
        lc.Creature.Species = sp
        dc := render.NewCard(DemoUser, s, render.Dark)
        dc.Creature.Species = sp
        if err := write(f.Light, render.SVG(lc)); err != nil {
            return f, err
        }
        if err := write(f.Dark, render.SVG(dc)); err != nil {
            return f, err
        }
        f.Alt = "commitling: " + render.Description(lc)
        return f, nil
    }

    p := page{
        Version:    version,

```

```

        Title:      Title,
        Description: Description,
        SiteURL:    SiteURL,
        OGImage:    OGImage,
        LiveUser:   LiveUser,
        Workflow:   WorkflowSnippet,
        Readme:     ReadmeSnippet,
        ReadmeDrk:  `<picture>
<source media="(prefers-color-scheme: dark)" srcset="./commitling-dark.svg">

</picture>`,
    }

    extras := []struct {
        name, caption string
        s              stats.Stats
    }{
        {"acc-hat", "Gorro: más de 30 días activos en 90", stats.Stats{XP: 700, DaysSinceLast: 0, Streak: 3,
        ActiveDays30: 20, ActiveDays90: 42, Repos: 3}},
        {"acc-scarf", "Bufanda: racha de 10 días o más", stats.Stats{XP: 1500, DaysSinceLast: 0, Streak: 12,
        ActiveDays30: 21, ActiveDays90: 28, Repos: 4}},
        {"acc-flower", "Flor: 5 repositorios distintos o más", stats.Stats{XP: 260, DaysSinceLast: 1, Streak: 2,
        ActiveDays30: 12, ActiveDays90: 20, Repos: 6}},
        {"acc-all", "Todo desbloqueado", stats.Stats{XP: 3900, DaysSinceLast: 0, Streak: 14, ActiveDays30: 27,
        ActiveDays90: 64, Repos: 9}},
    }

    intros := map[creature.Species]string{
        creature.MossSprout: "La especie predeterminada: un brote de musgo con ojos que acaba siendo un árbol
        ancestral.",
        creature.Mushroom:  "Una seta pequeña con ojos que empieza siendo una espora y acaba en un corro de setas.
        Se elige con <code>species: mushroom</code>.",
    }

    for _, sp := range creature.AllSpecies {
        // The default species keeps the file names of the first release.
        prefix := ""
        if sp != creature.MossSprout {
            prefix = sp.Slug() + "-"
        }
        sec := speciesSection{ID: sp.Slug(), Name: sp.Name(), Intro: template.HTML(intros[sp])}
        for _, st := range creature.Stages {
            r := row{Title: fmt.Sprintf("%s · desde %s XP", sp.StageName(st), render.Thousands(st.MinXP()))}
            for _, m := range creature.Moods {
                f, err := card(prefix+st.Slug()+"-"+m.Slug(), sp, Sample(st, m))
                if err != nil {
                    return n, err
                }
                f.Caption = m.Name()
                r.Figures = append(r.Figures, f)
            }
            sec.Rows = append(sec.Rows, r)
        }
        for _, e := range extras {
            f, err := card(prefix+e.name, sp, e.s)
            if err != nil {
                return n, err
            }
            f.Caption = e.caption
            sec.Extras = append(sec.Extras, f)
        }
        p.Species = append(p.Species, sec)
    }

    icon := render.SpriteSVG(creature.Creature{Stage: creature.Sprout, Mood: creature.Happy})
    if err := write("favicon.svg", icon); err != nil {
        return n, err
    }
    p.Favicon = "favicon.svg"
    for _, sp := range creature.AllSpecies {
        p.GenSpecies = append(p.GenSpecies, choice{sp.Slug(), sp.Name()})
    }
    if err := write("generator.js", generatorJS); err != nil {
        return n, err
    }
    hero := render.SpriteSVG(og.Hero)
    if err := write("hero.svg", hero); err != nil {
        return n, err
    }
}

```

```

p.Hero = "hero.svg"
card1200, err := og.PNG(og.Hero)
if err != nil {
    return n, err
}
if err := write("og.png", card1200); err != nil {
    return n, err
}

p.XP = []rule{
    {"Commit", fmt.Sprintf("%d XP", stats.XPCommit)},
    {"Pull request abierta", fmt.Sprintf("%d XP", stats.XPPullRequest)},
    {"Issue abierta", fmt.Sprintf("%d XP", stats.XPIssue)},
    {"Otra actividad pública (estrellas, forks, comentarios...)", fmt.Sprintf("%d XP", stats.XPOther)},
}
for _, st := range creature.Stages {
    names := make([]string, len(creature.AllSpecies))
    for i, sp := range creature.AllSpecies {
        names[i] = sp.StageName(st)
    }
    p.Stages = append(p.Stages, rule{strings.Join(names, " / "), "desde " + render.Thousands(st.MinXP()) + " XP"})
}
p.Moods = []rule{
    {creature.Sleeping.Name(), fmt.Sprintf("%d días o más sin actividad (o ninguna)", creature.SleepingAfterDays)},
    {creature.Bored.Name(), fmt.Sprintf("de %d a %d días sin actividad", creature.BoredAfterDays, creature.SleepingAfterDays-1)},
    {creature.Happy.Name(), fmt.Sprintf("actividad en los últimos %d días", creature.BoredAfterDays)},
    {creature.Radiant.Name(), fmt.Sprintf("racha de %d días seguidos o más", creature.RadiantStreak)},
}
p.Acc = []rule{
    {"Gorro", fmt.Sprintf("más de %d días activos en los últimos 90", creature.HatActiveDays90)},
    {"Bufanda", fmt.Sprintf("racha de %d días o más", creature.ScarfStreak)},
    {"Flor", fmt.Sprintf("%d repositorios distintos o más con commits, PR o issues", creature.FlowerRepos)},
}

tmpl, err := template.New("index").Parse(indexTpl)
if err != nil {
    return n, err
}
var buf bytes.Buffer
if err := tmpl.Execute(&buf, p); err != nil {
    return n, err
}
html := strings.ReplaceAll(buf.String(), "\r\n", "\n")
if err := write("index.html", []byte(html)); err != nil {
    return n, err
}
return n, nil
}

```

internal/gallery/generator.js · 226 líneas

```
// Live generator of the website. The drawing is done by the same Go code as
// the command line, compiled to WebAssembly (commitling.wasm); this file only
// downloads the public events of a user from api.github.com, hands them to the
// wasm and shows the result. No dependencies, no credentials, no HTML built
// from outside data.
(function () {
  'use strict';

  var root = document.getElementById('gen');
  if (!root) return;

  var API = 'https://api.github.com/users/';
  var PAGES = 3; // 3 pages of 100: the 300 events the API keeps (about 90 days)
  var TIMEOUT_MS = 15000; // same as the command line client
  var REUSE_MS = 60000; // the API caches for about a minute: do not ask again sooner
  var form = document.getElementById('gen-form');
  var userInput = document.getElementById('gen-user');
  var button = document.getElementById('gen-go');
  var statusEl = document.getElementById('gen-status');
  var errorEl = document.getElementById('gen-error');
  var img = document.getElementById('gen-img');
  var empty = document.getElementById('gen-empty');
  var caption = document.getElementById('gen-caption');
  var after = document.getElementById('gen-after');
  var dl = document.getElementById('gen-dl');
  var wf = document.getElementById('gen-wf');

  var busy = false;
  var current = null; // {user, events, at} of the last successful search
  var blobURL = '';

  root.hidden = false;

  if (typeof WebAssembly !== 'object' || typeof fetch !== 'function') {
    errorEl.textContent = 'Tu navegador no admite WebAssembly, que el generador necesita. Prueba con una versión reciente de Firefox, Chrome, Safari o Edge.';
    button.setAttribute('aria-disabled', 'true');
    return;
  }

  function checked(name) {
    return form.elements[name].value;
  }

  function say(text) {
    errorEl.textContent = '';
    statusEl.textContent = text;
  }

  function fail(text) {
    statusEl.textContent = '';
    errorEl.textContent = text;
  }

  function setBusy(on) {
    busy = on;
    form.setAttribute('aria-busy', on ? 'true' : 'false');
    button.setAttribute('aria-disabled', on ? 'true' : 'false');
  }

  // The wasm and wasm_exec.js are only downloaded the first time they are
  // needed, which is when the person interacts with the generator.
  var loading = null;

  function loadScript(src) {
    return new Promise(function (resolve, reject) {
      var s = document.createElement('script');
      s.src = src;
      s.onload = resolve;
      s.onerror = function () { reject(new Error('no se pudo cargar ' + src)); };
      document.head.appendChild(s);
    });
  }

  function instantiate(go) {
    var url = 'commitling.wasm';
```

```

var viaBuffer = function () {
  return fetch(url).then(function (r) {
    if (!r.ok) throw new Error('commitling.wasm: ' + r.status);
    return r.arrayBuffer();
  }).then(function (b) { return WebAssembly.instantiate(b, go.importObject); });
};
if (!WebAssembly.instantiateStreaming) return viaBuffer();
// Falls back to a plain download when the server does not send application/wasm.
return WebAssembly.instantiateStreaming(fetch(url), go.importObject).catch(viaBuffer);
}

function ensureWasm() {
  if (!loading) {
    loading = loadScript('wasm_exec.js').then(function () {
      var go = new Go();
      return instantiate(go).then(function (result) {
        go.run(result.instance).catch(function () {});
        if (!window.commitling) throw new Error('el generador no se inició');
      });
    });
    loading.catch(function () { loading = null; }); // allow trying again
  }
  return loading;
}

// A signal that aborts the request after TIMEOUT_MS (none on old browsers).
function timeoutSignal() {
  return typeof AbortSignal !== 'undefined' && typeof AbortSignal.timeout === 'function'
    ? AbortSignal.timeout(TIMEOUT_MS) : undefined;
}

// Reads the public events of user: up to 3 pages, newest first. A failure
// is rejected as {status, remaining, reset}; status 0 means no answer at all.
async function fetchEvents(user) {
  var all = [];
  for (var page = 1; page <= PAGES; page++) {
    var res;
    var batch;
    try {
      res = await fetch(API + encodeURIComponent(user) + '/events/public?per_page=100&page=' + page,
        { headers: { Accept: 'application/vnd.github+json' }, signal: timeoutSignal() });
      if (res.status === 422 && page > 1) break; // past the last page
      if (!res.ok) {
        throw { status: res.status, remaining: res.headers.get('X-RateLimit-Remaining'), reset: res.headers.get('X-RateLimit-Reset') };
      }
      batch = await res.json();
      if (!Array.isArray(batch)) throw { status: 502 }; // a 200 that is not a list of events
    } catch (e) {
      throw e && typeof e.status === 'number' ? e : { status: 0 };
    }
    all = all.concat(batch);
    if (batch.length < 100) break;
  }
  return all;
}

function show(result, user, noActivity) {
  var url = URL.createObjectURL(new Blob([result.svg], { type: 'image/svg+xml' }));
  img.src = url;
  if (blobURL) URL.revokeObjectURL(blobURL);
  blobURL = url;
  img.alt = 'commitling de @' + user;
  img.hidden = false;
  empty.hidden = true;
  caption.textContent = '@' + user + ': ' + result.description;
  // The file is the SVG on screen; user has passed commitling.check, so the
  // name only holds letters, digits and dashes.
  dl.href = url;
  dl.download = 'commitling-' + user + '.svg';
  var w = window.commitling.workflow(user, checked('species'), checked('theme'));
  wf.textContent = w.error ? '' : w.workflow;
  after.hidden = !!w.error;
  say(noActivity
    ? '@' + user + ' no tiene actividad pública en los últimos 90 días: su criatura duerme.'
    : 'Listo: esta es la criatura de @' + user + '.');
}

```

```

function redraw() {
  var r = window.commitling.render(JSON.stringify(current.events), current.user, checked('species'), checked(
'theme'));
  if (r.error) {
    fail(r.error);
    return;
  }
  show(r, current.user, current.events.length === 0);
}

async function draw() {
  if (busy) return;
  var user = userInput.value.trim().replace(/^@/, '');
  if (!user) {
    fail('Escribe tu usuario de GitHub.');
```

userInput.focus();

return;

}

setBusy(true);

say('Cargando el generador...');

try {

 try {

 await ensureWasm();

 } catch (e) {

 fail('No se pudo cargar el generador. Comprueba tu conexión e inténtalo de nuevo.');

 return;

 }

 var invalid = window.commitling.check(user, checked('species'), checked('theme'));

 if (invalid) {

 fail(invalid);

 userInput.focus();

 return;

 }

 var events;

 if (current && current.user.toLowerCase() === user.toLowerCase() && Date.now() - current.at < REUSE_MS) {

 events = current.events; // same user a moment ago: nothing new to ask GitHub

 } else {

 say('Buscando la actividad pública de @' + user + '...');

 try {

 events = await fetchEvents(user);

 } catch (f) {

 fail(window.commitling.explain(f.status, f.remaining, f.reset));

 return;

 }

 current = { user: user, events: events, at: Date.now() };

 }

 current.user = user;

 redraw();

} finally {

 setBusy(false);

}

}

form.addEventListener('submit', function (e) {

 e.preventDefault();

 draw();

});

// Start downloading as soon as someone types in the form (submit and change

// load it too); merely tabbing through the page does not.

form.addEventListener('input', function () {

 ensureWasm().catch(function () {});

});

// Species and theme redraw what is already on screen, without asking GitHub again.

form.addEventListener('change', function (e) {

 if (e.target.type !== 'radio' || busy) return;

 ensureWasm().then(function () {

 if (current) redraw();

 }, function () {

 fail('No se pudo cargar el generador. Comprueba tu conexión e inténtalo de nuevo.');

 });

});

action.yml · 87 líneas

```
name: commitling
description: Dibuja una mascota pixel-art que crece con tu actividad pública de GitHub (SVG animado para tu README).
author: BertMarti
branding:
  icon: feather
  color: green

inputs:
  user:
    description: Usuario de GitHub cuya actividad pública se dibuja.
    required: false
    default: ${ github.repository_owner }
  out:
    description: Ruta del SVG de salida, relativa al repositorio.
    required: false
    default: commitling.svg
  theme:
    description: Tema de la tarjeta, light (papel) o dark (tinta).
    required: false
    default: light
  token:
    description: Token para la API de GitHub. Basta con el GITHUB_TOKEN del workflow.
    required: false
    default: ${ github.token }
  species:
    description: Especie de la criatura, moss (brote de musgo, la predeterminada) o mushroom (hongo).
    required: false
    default: moss
  fixture:
    description: Archivo JSON de eventos (relativo al repositorio) para trabajar sin red. Si se indica, no se llama a la API.
    required: false
    default: ""
  keep-on-error:
    description: Ante un fallo transitorio de la API (red, 5xx, límite de peticiones), conserva el SVG de salida existente y avisa (true o false).
    required: false
    default: "true"

outputs:
  path:
    description: Ruta absoluta del SVG generado.
    value: ${ steps.render.outputs.path }

runs:
  using: composite
  steps:
    # github.action_path is read from the environment on purpose: inside a
    # composite action it is only reliable in `run` steps.
    - id: paths
      shell: bash
      run: echo "gomod=$GITHUB_ACTION_PATH/go.mod" >> "$GITHUB_OUTPUT"

    - uses: actions/setup-go@v7
      with:
        go-version-file: ${ steps.paths.outputs.gomod }
        cache: false

    - id: render
      shell: bash
      env:
        GITHUB_TOKEN: ${ inputs.token }
        CL_USER: ${ inputs.user }
        CL_OUT: ${ inputs.out }
        CL_THEME: ${ inputs.theme }
        CL_SPECIES: ${ inputs.species }
        CL_FIXTURE: ${ inputs.fixture }
        CL_KEEP_ON_ERROR: ${ inputs.keep-on-error }
      run: |
        set -euo pipefail
        abs() { case "$1" in /*|/[A-Za-z]*) printf '%s' "$1" ;; *) printf '%s/%s' "$GITHUB_WORKSPACE" "$1" ;; esac; }
        out="$(abs "$CL_OUT")"
        args=(render --theme "$CL_THEME" --species "$CL_SPECIES" --out "$out")
        if [ -n "$CL_FIXTURE" ]; then
          args+=(---fixture "$(abs "$CL_FIXTURE")")
          if [ -n "$CL_USER" ]; then args+=(--user "$CL_USER"); fi
```

```

else
  args+=(--user "$CL_USER")
fi
# Composite inputs are always strings: accept only true or false.
case "$CL_KEEP_ON_ERROR" in
  true) args+=(--keep-on-error) ;;
  false) ;;
  *) echo "::error::keep-on-error debe ser true o false, no '$CL_KEEP_ON_ERROR'"; exit 2 ;;
esac
cd "$GITHUB_ACTION_PATH"
go run ./cmd/commitling "${args[@]}"
echo "path=$out" >> "$GITHUB_OUTPUT"

```

.github/workflows/ci.yml · 76 líneas

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    name: test (${{ matrix.os }})
    strategy:
      fail-fast: false
    matrix:
      os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}
    defaults:
      run:
        shell: bash
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-go@v7
        with:
          go-version-file: go.mod
          cache: false

      - name: gofmt
        if: runner.os == 'Linux'
        run: |
          unformatted="$(gofmt -l .)"
          if [ -n "$unformatted" ]; then
            echo "Archivos sin formatear:"
            echo "$unformatted"
            exit 1
          fi

      - name: go vet
        run: go vet ./...

      - name: go test
        run: go test ./...

      # The browser build: it must compile, its layer over internal/generate is
      # tested inside a real WebAssembly runtime (Node, preinstalled on the
      # runner) and its weight is shown in the log.
      - name: wasm
        if: runner.os == 'Linux'
        run: |
          GOOS=js GOARCH=wasm go vet ./cmd/wasm
          GOOS=js GOARCH=wasm go test -exec="$(go env GOROOT)/lib/wasm/go_js_wasm_exec" ./cmd/wasm
          GOOS=js GOARCH=wasm go build -ldflags="-s -w" -o out/commitling.wasm ./cmd/wasm
          echo "commitling.wasm: $(wc -c < out/commitling.wasm) bytes, $(gzip -9 -c out/commitling.wasm | wc -c)
gzipped"

      - name: render con el fixture
        run: |
          go run ./cmd/commitling render --fixture testdata/events.json --out out/commitling.svg
          go run ./cmd/commitling render --fixture testdata/events.json --theme dark --out out/commitling-dark.svg
          test -s out/commitling.svg
          test -s out/commitling-dark.svg

      - name: galería
        run: |
          go run ./cmd/commitling gallery --out out/site
          test -s out/site/index.html
          ls out/site/svg | wc -l

      - name: versión
        run: go run ./cmd/commitling version
```


.github/workflows/deploy.yml · 79 líneas

```
name: Pages

on:
  push:
    branches: [main]
  workflow_dispatch:
  schedule:
    - cron: "17 6 * * *" # cada día a las 06:17 UTC

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

env:
  LIVE_USER: BertMarti

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-go@v7
        with:
          go-version-file: go.mod
          cache: false

      - name: Galería
        run: go run ./cmd/commitling gallery --out site

      # The live generator: the same Go code as the CLI, compiled to WebAssembly.
      # wasm_exec.js must come from the same Go that compiles the wasm. Neither
      # file is in the repository.
      - name: Generador (WebAssembly)
        run: |
          GOOS=js GOARCH=wasm go build -ldflags="-s -w" -o site/commitling.wasm ./cmd/wasm
          cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" site/wasm_exec.js
          echo "commitling.wasm: $(wc -c < site/commitling.wasm) bytes, $(gzip -9 -c site/commitling.wasm | wc -c)
gziped" | tee -a "$GITHUB_STEP_SUMMARY"

      - name: Criatura en vivo
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          mkdir -p site/live
          go build -o "$RUNNER_TEMP/commitling" ./cmd/commitling
          cl="$RUNNER_TEMP/commitling"
          if "$cl" render --user "$LIVE_USER" --out "site/live/$LIVE_USER.svg" \
            && "$cl" render --user "$LIVE_USER" --theme dark --out "site/live/$LIVE_USER-dark.svg"; then
            echo "Criatura en vivo de @$LIVE_USER generada con datos reales." >> "$GITHUB_STEP_SUMMARY"
          else
            # The API failed: keep the site alive with the fictitious fixture
            # (the card then says @octoexample, so it is not passed off as real).
            echo "::warning::No se pudo leer la API de GitHub; la criatura en vivo usa el fixture de ejemplo."
            echo "API de GitHub no disponible: la criatura en vivo usa el fixture de ejemplo (@octoexample)." >>
"$GITHUB_STEP_SUMMARY"
            "$cl" render --fixture testdata/events.json --out "site/live/$LIVE_USER.svg"
            "$cl" render --fixture testdata/events.json --theme dark --out "site/live/$LIVE_USER-dark.svg"
          fi

      - uses: actions/configure-pages@v6

      - uses: actions/upload-pages-artifact@v5
        with:
          path: site

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
```

```
url: ${ steps.deployment.outputs.page_url }
steps:
  - id: deployment
    uses: actions/deploy-pages@v5
```